

Assignment

Deep Learning Architecture for ECG Heartbeat Classification: A study of CNN and Hybrid Models.

Module Code: CT100-3-M-DL
Module Name: Deep Learning

Intake Code: APUMF2508DSBA(DE)

Programme: Masters in Data Science and Business Analytics.

Submitted By:

Student Name 1: Tashfia Choudhury.

TP Number: TP091157.

Submitted To:

Module Lecturer: Dr. Raja Rajeswari.

Table of Contents

Abstract..... 4

1. Introduction & Background..... 5

2. Domain Knowledge and Problem Statement..... 8

2.1 Domain Background of ECG and Cardiac Arrhythmia..... 8

2.2 Problem Statement..... 9

2.3 Aim & Objective..... 10

2.3.1 Aim..... 10

2.3.2 Objective..... 10

2.4 Significance of the study..... 11

3. Literature and Justification of the Selected Deep Learning Algorithms..... 13

3.1 Formulation of ECG Heartbeat Classification..... 13

3.2 Review of Deep Learning Architectures for Classifying ECG..... 14

3.2.1 1D CNN..... 14

3.2.2 CNN-LSTM & CNN-BiLSTM..... 15

3.2.3 1D ResNet..... 17

3.2.4 Other Recent Architecture..... 18

3.3 Critical Analysis of Previous Studies..... 19

3.4 Comparative Summary..... 20

3.5 Research Gap & Justification..... 22

3.6 Selective Algorithm summary..... 23

4. Discussion of Methods..... 25

4.1 Research Methodology..... 25

4.2 Description of the Dataset..... 26

4.3 Dataset Preprocessing..... 27

4.3.1 Input Representation..... 27

4.3.2 Normalization..... 27

4.3.3 Label Encoding..... 28

4.4 Deep Leaning Models..... 28

4.4.1 Baseline 1D CNN..... 28

4.4.2 CNN-BiLSTM..... 30

4.4.3 1D ResNet..... 30

4.5 Model Training..... 32

4.6 Hyperparameter Tuning of 1D ResNet..... 32

4.7 Evaluation Metrics..... 33

4.7.1 Accuracy..... 33

4.7.2 Precision..... 34

4.7.3 Recall…… 34
4.7.4 F1-Score…… 34
4.7.5 ROC AUC……. 34
4.7.6 Confusion Matrix…….. 35
4.8 Model Comparison and Selection…… 35
4.9 Summary……. 36
5. Model Implementation……... 37
5.1 Data Preprocessing and preparation…… 37
5.2 Handling Class Imbalance…….. 40
5.3 Training/Test Split…….. 41
5.4 1D CNN Model (Baseline)…….. 42
5.5 CNN-BiLSTM Model……..... 44
5.6 1D ResNet Model…… 47
5.7 Model Evaluation…… 50
5.8 Architectural Modifications and Original Contributions…… 50
5.9 Summary……. 51
6. Hyperparameter Tuning, Validation and Final Evaluation……. 52
6.1 Hyperparameter Tuning Strategy…… 52
6.2 Tuning Results and Model Selection……... 53
6.3 Final Tuned 1D ResNet …….. 54
6.4 Validation and Final Evaluation…….. 55
7. Visualization & Critical Analysis……... 57
7.1 Comparative Performance Analysis…….. 57
7.2 Class-Level Performance Analysis…… 58
7.3 Critical Analysis of Model Performance…… 60
7.3.1 Comparison of the Three Initial Architectures……. 60
7.3.2 Effects of Hyperparameters Tuning…… 60
7.4 Interpretation of ROC-AUC……. 61
7.5 Strengths and Limitations……. 62
7.5.1 Strengths……. 62
7.5.2 Limitations…….. 63
7.6 Overall Critical Finding……. 63
7. Conclusion……. 65
8. References……..... 66

Abstract

A serious healthcare concern is Cardiovascular illness and for tracking its electrical activity of the heart and detecting abnormal cardiac rhythms Electrocardiography (ECG) is a significant non-operative method. But understanding the ECG signals manually is time-consuming and might end up affected by the complex nature and diversity of cardiac patterns. Here, this study implementing different type of deep learning methods for computerized ECG heartbeat classification by using an ECG Heartbeat Categorization Dataset from Kaggle which is collected from two famous databases MIT-BIH Arrhythmia Database and The PTB Diagnostic ECG Database. Three different deep learning models: 1-dimensional convolutional neural network (1D CNN), combined CNN with bidirectional long-short term memory (CNN-BiLSTM) and one-dimensional residual network (1D ResNet) are implemented and compared, and lastly the top-performing model is optimized by using hypermeter tuning and evaluated all 4 models using accuracy, recall, precision, Macro-F1 score and ROC-AUC. The study fits with the previous research work that shows the performance of deep learning and residual convolutional models for ECG and provides comparative assessment of different types of architectures and a tuned 1D ResNet model for the heartbeat classification problem.

Keywords – ECG Heartbeat Classification, Deep Learning, 1D CNN, CNN-BiLSTM, 1D ResNet, Hyperparameter Tuning, ECG Signals, Arrhythmia Detection, MIT-BIH Arrhythmia Dataset, Automated Cardiac Diagnosis.

1. Introduction & Background

Cardiovascular disease is a common health condition all around the world and for its diagnosis cardiac arrythmias is an important factor. Arrythmias means abnormal heartbeat rhythms which are categorized as moderate to severe conditions. Electrocardiography (ECG) is the main non-operative method for tracking electrical activity of heart because of comprehensive time-related data and affordability. Due to massive growth of digital ECG records, it has become possible for computational techniques to improve signal analysis and classification. According to Safdar et al. (2024), recent research works using deep learning methods more to develop an automated ECG interpretation and identifying arrhythmias system.

ECG signals capture the electrical activity of the heart which looks like unique waveform patterns that represent different cardiac stages. People detect between normal and irregular heartbeat using the changes in waveform. Traditional ECG analysis used to need signal processing and manual feature designing which required in-depth domain knowledge. According to Safdar et al. (2024) highlighted machine learning and deep learning techniques like recurrent and convolutional models have become popular for this type of work. This change was needed because these techniques can direct generation of hierarchical representations from ECG data which reduces the need for manual feature engineering.

Deep learning works better for classifying ECG heartbeats because ECG data are sequential. Both waveform temporal and morphology patterns are important. For example, 1D CNN uses convolutional filters for finding the local patterns in the data which works effectively in ECG classification. In a study of Kim et al. (2023), proposed a CNN architecture named WavelNet for arrhythmia classification and similarly Yang et el. (2021) proposed cascaded CNN for 12 lead ECG analysis. Both studies demonstrated the broad scope of convolutional techniques.

Another architecture which combines CNN with bidirectional long-short term memory (CNN-BiLSTM) is known as promising method for analyzing ECG signals. Here CNNs extract morphological features and BiLSTM analyzes sequential data by processing in both directions for improving contextual understanding. Hassan et al. (2022) highlighted the potential of hybrid models and showed effectiveness of CNN-BiLSTM architecture in classifying cardiac arrhythmias from ECG data.

Lastly, residual neural networks (ResNets) become the best architecture for ECG classification as it has improved the classification by using shortcut connections for more effective information flow which resolved the optimization problems in deep learning networks. In a study of Jing et al. (2021), it achieved better results for ECG classification by using an advanced ResNet-18 on the MIT-BIH arrhythmia database. Also, in another study of Khan et al. (2023), developed a 1D convolutional deep residual neural network which performed excellent on a range of evaluation metrics. With all these studies, it proves the potential of 1D ResNet architecture for classifying ECG heartbeat.

Selecting the type of architecture for classifying ECG heartbeat is very critical because overall high accuracy might mislead due to class imbalance in datasets. Sometimes large number of normal heartbeats data leads to potential underperformance in detecting minor abnormal categories. For that reason, recent studies use a range of evaluation metrics like F1 score, precision and ROC-AUC etc. Both Hassan et al. (2022) and Khan et al. (2023) implemented multiple metrics in their study. Therefore, this study focuses on macro-averaged metrics to ensure accurate performance for multiple heartbeat classes.

Every research highlighted that there isn't single deep learning model that works best for classifying ECG heartbeats, there are many like CNN, recurrent network, CNN-RNN and intricate attention-based architectures. According to Bai et al. (2024), they developed hybrid CNN, bidirectional gated recurrent units and multi head attention for diagnosing cardiac arrhythmia automated. On another study of Islam et al. (2023), they implemented hybrid model of dilated CNNs and hierarchical attention based bidirectional recurrent networks. All of these studies indicate that model setup, architecture options and training approach puts major impact on classification performance.

This project implements a comparative experimental framework with three deep learning models: Basic 1D-CNN, Hybrid CNN-BiLSTM and 1D ResNet to examine ECG heartbeat classification in healthcare. Then the strongest model is used for further optimization once all models are done training and evaluation consistently. Hyperparameter like dropout, learning rate and network size etc tuning are done for improving the 1D ResNet model's performance. Finally, evaluation metrics like accuracy, macro precision, macro F1-score, macro recall and macro-ROC-AUC are used to evaluate the performance of the customized model in comparison to the original design.

The purpose of this study is to investigate how well different deep-learning models 1D-CNN, Hybrid CNN-BiLSTM and 1D ResNet perform for the ECG heartbeat classification. It also examines how hyperparameter tuning tackles issues like class imbalance and impacts on classification accuracy. The aim is to design the best deep-learning architecture for this system which can improve the effectiveness of automated ECG classification systems.

2. Domain Knowledge and Problem Statement

2.1 Domain Background of ECG and Cardiac Arrhythmia

The primary focus of this study is to implement automatic cardiac arrhythmias classification and cardiovascular signal analysis. It emphasizes the importance of monitoring cardiac signals (ECG) which capture electrical activity of heart virtually (visual of ECG signal is shown in **Figure 2.1.1** with code). ECG provides waveforms forms that are essential for investigating features like morphology, intervals and rhythms patterns which are useful for the classification of both normal and pathological heartbeats.

```
fig, axes = plt.subplots(num_classes,1,figsize=(14, 14))

for class_id in range(num_classes):indices = np.where(y_train_full == class_id)[0]

    sample_index = indices[0]

    axes[class_id].plot(X_train_full[sample_index])

    axes[class_id].set_title(f"Example ECG Signal - {class_names[class_id]}")

    axes[class_id].set_xlabel("Time / Sample")
    axes[class_id].set_ylabel("Amplitude")

plt.tight_layout()
plt.show()
```

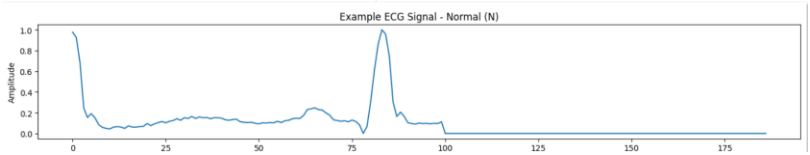


Figure 2.1.1: Representation of ECG signal using the dataset.

Irregular heartbeat rate is a key concept in cardiac arrhythmia. Arrhythmias can develop due to the changes in the form or timing of the heartbeat. The purpose of automated ECG analysis advances beyond identifying ECG signals simply by incorporating heartbeats classification and detecting meaningful patterns. Safdar et al. (2024) highlighted the rapid growth of using artificial intelligence and deep learning in ECG analysis because of their capacity to learn from physiological signals which reduces the dependence on manually engineered patterns.

The number of ECG signals has been growing rapidly, which creates computational challenges that make manual analysis unfeasible. Only automated classification systems can process and categorize the ECG signals effectively that help the physicians and academics to analyze big datasets. Even so, these systems should be used as decision-support tools rather than using as clinical diagnosis replacements, expert interpretation requires additional patient context and assessment.

Deep learning works better for classifying ECG heartbeats because ECG data are sequential which relies on temporal patterns and local waveform properties for classification. 1D-CNN detects local patterns while LSTM networks manage dependencies in sequential data. So, combining both of them into one framework makes use of bidirectional temporal modeling along with local feature extraction. According to researchers (Zhang et al. (2021), Hassan et al. (2022), Kim et al. (2023)) CNN and CNN-BiLSTM models are the most effective in classifying ECG arrhythmia.

On the other hand, residual learning provides an advanced approach to extract complicated patterns from ECG data by using ResNet architectures. This enables the utilization of shortcut connections that improve information and gradient propagation. Because of its efficiency, it makes residual architecture more suitable because it enables the network to learn complex signal representations beyond convolutional layers for ECG classification. As a result, researchers (Jing et al. (2021), Khan et al. (2023)) highlighted the effectiveness of improved ResNet models and 1D deep residual networks in classifying ECG heartbeats. Furthermore, Bai et al. (2024) explored hybrid architectures that combine convolutional and recurrent methods with careful mechanisms which advances deep learning techniques for automated ECG analysis.

2.2 Problem Statement

The purpose of this study is to build multi-class ECG classification model by using deep leaning approaches. The project ECG heartbeat data contains five classes: Normal (N), Supraventricular (S), Ventricular (V), Fusion (F) and Unknow (Q). The goal of the model is to learn from segmented ECG heartbeat signals and classifying each heartbeat to its corresponding class. These categories reflect various heartbeat patterns and provide multi-class classification challenges instead of a straightforward binary normal-abnormal prediction test.

The major problem in this system is the unequal distribution of heartbeat classes where normal heartbeats are far more common than many abnormal types. This gap makes a model to perform poor on minority classes while attaining high accuracy overall. So, using accuracy only as evaluation metric is insufficient to evaluate a model. That's why this study uses multiple evaluation metrics: accuracy, recall, F1-score, precision and ROC-AUC so that it can ensure

each class is given equal weight while preventing the majority class's performance from overshadowing the evaluation.

Then next difficulty is choosing the best deep learning architecture for ECG signals. A basic CNN can identify local patterns, but it can have problems in understanding sequential associations. On the other hand, combining CNN-BiLSTM model is good for modeling temporal information but it may not perform better than other architecture for all ECG classification tasks. 1D ResNet provides another technique by using residual connections for deep feature learning. So, the goal of this study is to compare different deep learning architectures within a unified evaluation framework.

To conclude, this study addresses this problem by developing and comparing three deep learning architectures: 1D CNN, CNN-BiLSTM and 1D ResNet. Then the strongest architecture is selected for hyperparameter tuning which focuses on training and architectural parameters like dropout rate, learning rate and network size. Finally, the tuned 1D ResNet model is evaluated against the original architectures of all models using consistent metrics.

2.3 Aim & Objectives

2.3.1 Aim

The aim of the project is to identify an appropriate architecture for automated ECG heartbeat classification by developing, comparing and optimizing deep learning models so that it can provide accurate and balanced multi-class classification of ECG heartbeats.

2.3.2 Objectives

For accomplishing the aim of the study, the project is divided to the following precise objectives:

a. **Objective 1: Prepare and analyze the ECG dataset**

   To examine the class distribution, preprocessing input signals for deep learning, splitting datasets into training, testing and validation partitions and addressing the effect of class imbalance.

b. **Objective 2: Develop a baseline 1D CNN model**

To develop a 1D CNN model to learn local morphological patterns from ECG data and create baseline performance for classification.

c. **Objective 3: Develop CNN-BiLSTM model**

To develop a combine CNN-BiLSTM model for ECG heartbeat classification by combining bidirectional processing and convolutional feature extraction which allows the study to see whether incorporating sequential information for improving ECG heartbeat classification.

d. **Objective 4: Develop 1D ResNet model**

For investigating alternative options for ECG heartbeat classification, develop 1D ResNet model that uses residual connections and deeper feature extraction.

e. **Objective 5: Compare all deep learning models**

Compare the performance of 1D CNN, CNN-BiLSTM and 1D ResNet models by using evaluation metrics accuracy, recall, precision, F1-score and ROC-AUC, with specific emphasis on the differences among minority and majority heartbeat classes.

f. **Objective 6: Apply hyperparameter tuning in the best-performing Model**

Identify the top-performing model and apply hyperparameter tuning to maximize validation performance while ensuring generalization.

g. **Objective 7: Analysis critically the best model**

Analyze the best model among all four models using ROC curves, confusion metrics, classification reports and conduct visual comparisons on an independent test ser.

2.4 Significance of the Study

This study is not focused on only one single model; it emphasizes a comparative evaluation of multiple deep learning architecture for classification of ECG heartbeats. According to Khan et al. (2023), variety of models including CNN, hybrid, recurrent and residual models can provide better results. But the effectiveness of these models fully depends on the selection of dataset, training strategies, architecture and evaluation metrics. The primary goal of this study is to compare different architectures based on evidence for understanding their advantages and disadvantages way better.

Another key component of this study is to address the class imbalance in ECG heartbeat classification. It emphasizes that strong performance in normal heartbeat classes overlooks the poor ability to identify less common abnormal classes. That's why this study provides a comprehensive balanced assessment of model performance by using metrics like accuracy and ROC-AUV along with precision, recall and F1-score which show flaws missed by accuracy alone.

This study also checks whether hyperparameter tuning of residual model can improve classification performance over the initial 1D ResNet model. It compares 1D-CNN, CNN-BiLSTM, 1D ResNet and a tuned 1D ResNet which showcase the progression from a basic convolutional model to a hybrid model. This comparison study identifies the top-performing model and examines how architectural layout and hyperparameter tuning options affect the performance results.

Day by day more physiological data is being produced by digital health technology, that's why an efficient automated ECG classification is needed because it can improve speed at which large amount of ECG data can be analyzed and can identify important heartbeat patterns. This study focuses on illustrating the application of deep learning-based ECG heartbeat classification rather than providing validated clinical diagnostic solutions.

In conclusion, this study provides a practical evaluation of multiple deep learning methods for classifying ECG classification where it emphasizes the importance of architecture selection, evaluation metrics, managing class imbalance and hyperparameter tuning in developing standard deep-learning algorithms for medical signal analysis.

3. Literature and Justification of the Selected Deep Learning Algorithms

3.1 Formulation of ECG Heartbeat Classification

This study manages a supervised multi-class time-series classification problem by using ECG data which captures the electrical activity signals of heart over time. The continuous ECG data is split into discrete heartbeat samples which are expressed as a numerical signal in one dimension. The main purpose of the algorithm is to find out patterns of each heartbeat and assign it to 1 to 5 predefined classes: Normal (N), Supraventricular (S), Ventricular (V), Fusion (F) and Unknow (Q) using ECG Heartbeat Categorization Dataset. In a study of Hassan et al. (2022), implemented the same five-categorized problem and evaluated CNN-BiLSTM model using MIT-BIH Arrhythmia Database. From a data science perspective, the below **equation (1)** can express the in a function:

$$f(X) = Y \qquad \text{.....(1)}$$

Where, X = ECG heartbeat signal

Y = Corresponding heartbeat class.

First the model needs to learn the mapping from labeled training instances so that it can generalize the unseen test samples. This is more difficult than simple binary classification. Because of the requirement to understand difference between several heartbeat types that may have similar morphological characteristics.

Class imbalance is the major challenge in ECG heartbeat dataset where the number of normal pulses outnumber certain abnormal pulse categories like supraventricular (S) and fusion (F). This can cause models with overall accuracy but poor performance in less frequent clinically classes. In a study of Khan et al. (2023) showed that they used SMOTE to reduce class imbalance before implementing their 1D residual network yet other studies have explored focusing on mechanisms and weighted loss functions to enhance minority class recognition.

One of the evaluation factors for classification performance is accuracy and it shouldn't be the only one. Capabilities for identifying model's class provided by recall, precision, F1-score and ROC-AUC. Macro-averaged metrics are specifically beneficial because they provide each class equal weight and prevent the dominant normal class from distorting the findings.

Deep learning is appropriate for analyzing ECG heartbeat signals because it can capture temporal associations and local morphological features. CNN can learn signal properties while recurrent architectures describe sequential dependencies. As residual configuration enables deeper convolutional feature extraction by using shortcut connections. And it makes it feasible to compare multiple deep learning models so that it can determine the best method for classifying ECGs.

3.2 Review of Deep Learning Architectures for Classifying ECG

3.2.1 1D CNN

1D CNN is one of the essential architectures for ECG analysis because it can directly work on sequential ECG data. Its convolutional filters extract unique patterns from the data during training and eliminates the need for manually generated features. Because of the sequential convolutional layers, these filters are able to process complicated data rapidly and can detect local differences in waveform patterns. In **equation 2** it shows the simple 1D CNN operation where CNN filters ECG data and learns local signal patterns (Cacciari & Ranfagni, 2026).

$$y_i = \sum_{k=0}^{k-1} x_i + kW_k \qquad \text{.....(2)}$$

(Cacciari & Ranfagni, 2026).

Where, x = Sequential ECG data

W_k= Filter weights

k = Kernal Index

y_i= Positioned output feature value

According to Yang et al. (2021), they used cascaded 1D CNN for classifying 12-lead ECG arrhythmia and they achieved 86.5% result in the China ECG intelligence Challenge. Even having the variations from the current heartbeat-class classification issue, their system showed effectiveness of convolutional feature extraction by combining characteristics from individual leads with expert features for ECG analysis.

On the other hand, Kim et al. (2023) created a WavelNet which is a special CNN architecture for ECG arrhythmia classification. This system proved how CNN architectures are still working better than traditional convolutional stacks.

So, the key benefit of 1D CNN is its ability to learn ECG morphology efficiently while maintaining very simple architecture. Though CNN has some drawbacks like their local receptive fields would not be able to reflect longer-range dependencies until network is deeper or includes processes like dilation significantly.

CNN contains both advantages and disadvantages, still it will be suitable for this study baseline architecture. Here it is not always assumed to offer the best accurate representation of all temporal data.

3.2.2 CNN-LSTM and CNN-BiLSTM

The creation of hybrid-CNN architecture because of the CNN's inability to modeling sequential dependencies. These hybrid model's recurrent layers often learn temporal associations in the extracted feature sequence after CNN layers are used for extracting local morphological features. On the other hand, LSTM models work better for sequential data due to their gating mechanisms which allow information to be stored or discarded over time. And BiLSTM is the extended version because it processes data in both forward and backward directions. Below **Figure 1 and Figure 2** show the architecture example of a CNN and BiLSTM (Hassan et al., 2022).

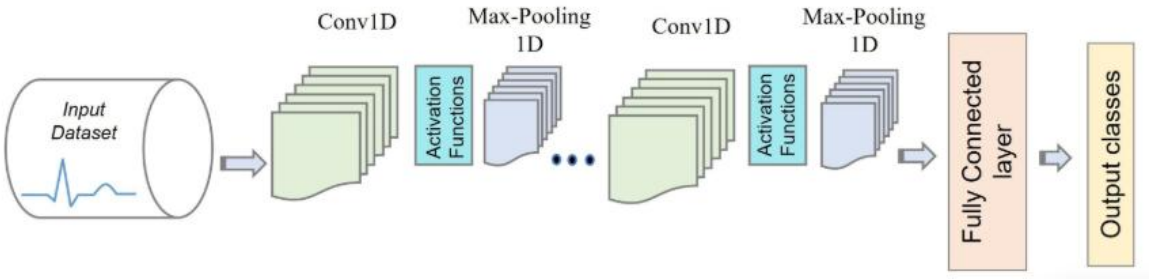


Figure 1: Architecture of CNN (Hassan et al., 2022).

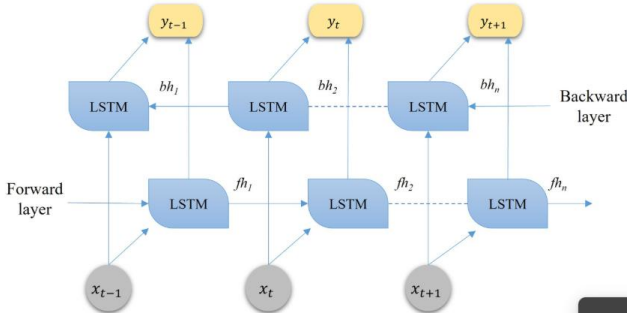


Figure 2: Architecture of BiLSTM (Hassan et al., 2022).

According to Hassan et al. (2022) work, they developed CNN-BiLSTM model for classifying five-class heartbeat using MIT-BIH dataset and their model achieved 100% training accuracy along with 98% validation accuracy and 98% testing accuracy. Most important part the study evaluated precision, recall, F1-score and specificity along with accuracy. Below **Figure 3** show the architecture example of a CNN-BiLSTM (Hassan et al., 2022).

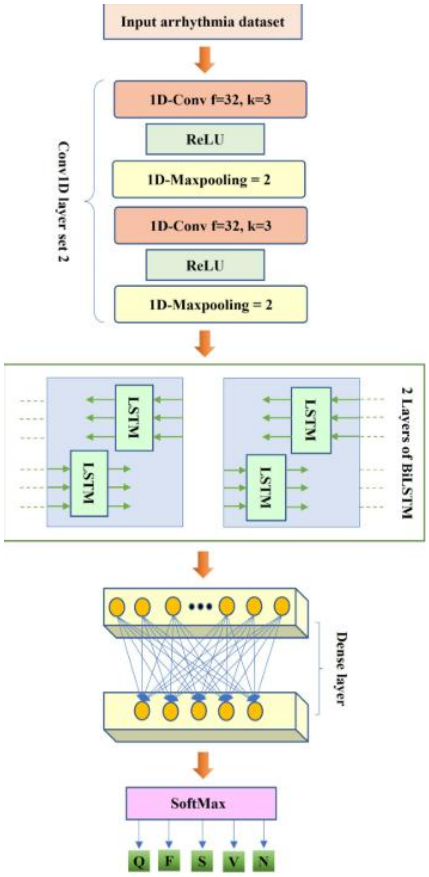


Figure 3: Architecture of CNN-BiLSTM (Hassan et al., 2022).

Similary, in another study of Yang et al. (2022), a lightweighted CNN-BiLSTM model with a loss function of weight was proposed for analyzing five-class heartbeat using MIT-BIH database. They checked results of accuracy (99.33%), sensitivity (93.67%), specificity (99.18%), F1-score (91.65%) and positive predictive value (89.85%) in order to reduce the effects of class imbalance.

These all-research studies support CNN-BiLSTM to be a comparative model in this study. Even though it needs higher training requirements and complexity. It is not always the suitable one for every dataset even if it performs well.

3.2.3 1D ResNet

Alternative option for increasing the depth of convolutional architecture is residual neural networks. Sometimes traditional deep architecture can have issues from optimization because of adding additional layers. But ResNet solve this problem by using shortcut connections which allows information and gradients to bypass one or more convolutional layers. In **equation 3** it shows the simple mathematical representation of 1D ResNet operation (Jing et al., 2021).

$$Y = F(x, \{w_i\}) + x \quad \ldots\ldots\ldots\

suitable for this study because it focuses on 1D residual architecture and the MIT-BIH classification issue.

According to the research evidence, it confirms that ResNet models are promising methods for classifying ECG data. It enables the development of deeper hierarchical visualization while keeping effective local feature extraction of CNN. All this offers a solid theoretical and experimental foundation for selecting 1D ResNet for this study's main architecture.

3.2.4 Other Recent Architectures

Research on ECG has evolved beyond conventional CNN and recurrent structure. Now it also uses attention mechanisms, hybrid networks, DenseNet-based models and multimodal techniques.

In research of Islam et al. (2023), they proposed HARDC which is a combined of dilated CNNs with bidirectional recurrent network and hierarchical attention so that it can capture both global and local data while highlighting limitations connected with conventional dilated CNNs. And achieved accuracy of 99.60%, F1-score of 98.21%, precision of 97.66% and recall of 99.60%.

In another research study of Bai et al. (2024) implemented CNN-BiGRU model which includes multi-head attention by using MIH-BIH arrhythmia database. They reported results of 99.41% accuracy, 99.21 F1-score, 99.15% precision and 99.68% specificity. Also, they achieved 98.82% accuracy on the PTB diagnostic ECG database. They highlighted the potential of attention mechanisms which can capture global association between different ECG classes efficiently.

Ullah et al. (2022) investigated and developed the DenseNet-based method by using MIT-BIH and INCART data generated beat images with stratified five-fold validation. It shows that dense connections work better to improve feature propagation in imbalanced ECG classification problems.

Then Zhou and Fang (2024) developed a multimodal architecture which converted ECG signal to recurrence plots, Markov transition fields and Gramian angular fields before

implementing CNN with a feature-channel attention mechanism. Their architecture performed 99.6% accuracy on five class arrhythmias in MIH-BIH. This shows the major change from processing 1D data to learning from several ECG morphological representations.

All of these improvements refer to a trend for more complex designs in the field. Though more complexity does not ensure that architecture is suitable for particular real-world problems. Advanced models might perform well but they will increase methodological complexity which will need more processing power. So, the purpose of this study is to build an architecture that maintains a balance between increasing complexity and useful implementation and analysis.

3.3 Critical Analysis of Previous Studies

There is not any particular model which always performs the best for classifying ECG heartbeats according to the previous studies. It is proven that to determine performance of a model, it depends on the architecture, class-imbalance solving, preprocessing, data segmentation and evaluation process.

On of the significant issue found from the literature is that class-imbalance. Hassan et al. (2022) performed assessments on several class-sensitive metrics while khan et al. (2023) applied SMOTE to boost minority class representation and it showed classification behavior can be enhanced by varying the contribution of multiple classes.

Another significant concern is generalization among patients. In a study of Yang et al. (2021), they examined interpatient pulse classification by using adversarial CNN which helped to achieve sensitivities of 78.8% for specific supraventricular ectopic and 92.5% for ventricular ectopic beats on MIH-BIH data from 47 patients. Its precisions were 90.8% and 94.3% respectively. So, this study highlighted the limitation fact that a model can perform well on a randomly dispersed heartbeat, but it struggles more when it is tested on patients who have never been seen before.

As a result, this is a crucial consideration while analyzing the outputs of this study. High accuracy of a model should not be taken as evidence for it being ready for clinical use. When

individual patients provide several heartbeat data, reported results can be significantly impacted by variations in train-test splitting methods. So, it is good for examining results from multiple research projects carefully rather than taking claimed accuracy scores.

According to the previous studies, simple CNN architectures have improved into hybrid and attention-based models. Though CNNs are great at learning local features, CNN-BiLSTM models can improve temporal visualization, and residual networks can work better in deeper feature extraction with better optimization. And attention models capture global dependencies. Safdar et al. (2024) has highlighted the rapid use of recurrent and transformer-based methods in ECG analysis.

The ideal architecture for this study might not match with highest reported accuracy architecture. Like in Bai et al (2024), they used CNN-BiGRU attention model to achieve 99.41% accuracy but here the architecture is way more complicated than traditional 1D ResNet. On the other hand, Zhou and Fang's (2024) used multimodal approach for converting 1D ECG signals to many image representations before performing classification.

Considering all, that's why this study focuses on controlled architecture comparison. It won't implement any complex architecture to increase accuracy. It will explore three fundamentally experimental models: 1D CNN, CNN-BiLSTM and 1D ResNet with consistent experimental structure. And it will be able explain whether more complex structure in recurrent processing is good over convolutional and residual feature learning for the selected dataset.

3.4 Comparative Summary

In **Table 1**, it shows the summary part of the most recent studies that are reviewed for this study. This study collected most of the research papers that are using the MIH-BIH arrhythmia database because it is most relevant to the problem of this study which is five class heartbeat classification.

Study	Dataset	Architecture / Method	Reported Results	Main Strength	Key Limitation / Consideration

Zhang et al. (2021)	MIT-BIH	Adversarial CNN.	SVEB sensitivity 78.8%, precision 90.8%, VEB sensitivity 92.5%, precision 94.3%.	Highlights interpatient variability.	Minority SVEB is still difficult.
Jing et al. (2021)	MIT-BIH	Improved ResNet-18.	Accuracy 96.50%, VEB sensitivity 93.83%, precision 97.44%.	Deeper residual feature learning.	Too complex than basic CNN.
Yang et al. (2021)	China ECG Intelligence Challenge	Cascaded 1D CNN, ResNet, expert features.	Final result 86.5% for 9 categories.	Combines learned and expert features.	Different multi-label 12-lead problem.
Hassan et al. (2022)	MIT-BIH and St. Petersburg	CNN-BiLSTM.	MIT-BIH test accuracy 98%.	Combines morphology and temporal information	Recurrent component increases complexity.
Cacciari and Ranfagni (2026)	MIT-BIH	Weighted CNN-BiLSTM.	Accuracy 99.33%, F1 91.65%, sensitivity 93.67.	Shows class imbalance using weighted loss.	Lower F1 than overall accuracy suggests minority-class difficulty.
Ullah et al. (2022)	MIT-BIH and INCART	2D DenseNet.	Stratified 5-fold evaluation.	Used dense feature propagation for imbalanced data.	Lower ECG-to-image conversion.
Khan et al. (2023)	MIT-BIH	1D Deep ResNet and SMOTE.	Accuracy 98.63%, F1 92.63%, sensitivity 92.41%.	Very relevant 1D residual method.	Needs oversampling strategy.
Kim et al. (2023)	ECG arrhythmia data	WavelNet CNN.	Strong CNN-based arrhythmia classification	Advanced CNN architecture.	More complex than baseline CNN.
Islam et al. (2023)	ECG datasets / MIT-BIH-derived data	Dilated CNN and BiGRU/BiLSTM and attention.	Accuracy 99.60%; F1 98.21%	Captures local and global information.	Too complex.
Bai et al. (2024)	MIT-BIH and PTB Diagnostic	CNN-BiGRU and multi-head attention.	MIT-BIH accuracy 99.41%, F1 99.21%.	Strong spatio-temporal and global modelling	Much architectural complexity.
Zhou & Fang (2024)	MIT-BIH	Multimodal CNN and FCA.	Accuracy 99.6%.	Uses multiple ECG visualization.	Needs conversion into multiple image modalities.

Table 1: Summary of the literature papers.

Above reported data should not be directly because of variations in preparation pipeline, assessment methodology, data partition and classification task. Like an example, Hassan et al. (2022) worked on five class heartbeat categories whereas Yang et al. (2021) used 9 category 12-lead ECG task.

From **Table 1**, it shows three major patterns:

1) They faced issues capturing long-term temporal relationships while traditional CNN architecture proved good at extracting ECG features.
2) CNN-BiLSTM models performed really well even when the complexity increased by combining sequential and morphological data.
3) Khan et al. (20023) reported 98.63% of accuracy by using 1D residual network highlights that residual architecture provide good blend of feature extraction, depth and optimization stability which relevant with the project's problem formulation.

At the same time, recent attention-based models have shown improved performance even with complex architectures. Bai et al. (2024) implemented CNN, BiGRU and multi-head attention to get 99.41% accuracy. Though, directly comparing straightforward models like 1D ResNet is not suitable without considering pre-processing, validation method, design and computational costs.

3.5 Research Gap & Justification

Though the reviewed research papers demonstrate significant advancements in automated ECG classification, the methodology used in this study is adopted by a number of observations.

Firstly, recent literature on hybrid models for classifying ECG heartbeat combines attention mechanisms, other feature-engineering methods and recurrent layers which achieved good results, but it made it difficult to identify the effort of each component. Secondly reported results sometimes cannot be comparable directly because of differences in preprocessing, class balancing methods, evaluation methods and train-test splits. Lastly, it is still beneficial to compare residual learning with basic CNN and CNN-BiLSTM models in the same framework even if ResNet- based methodss works well.

That's why this study suggests a controlled comparative analysis of three different structured models: 1D CNN, CNN-BiLSTM and 1D ResNet. 1D CNN will explain how well it can learn local ECG morphology and works as a basic convolutional baseline. Then CNN-BiLSTM contains bidirectional temporal modeling so that it can determine sequential data and offers extra prediction value. Lastly, a deeper convolutional method 1d ResNet uses residual connection for helping in feature propagation and optimization.

1D ResNet is selected as the main architecture for additional optimization because it is justified with both literature and the characteristics of the problem. Khan et al. (2023) demonstrated the effectiveness of 1D residual architecture on the MIH-BIH class heartbeat database which archived 98.63% of accuracy and 92.63% of F1-score. Additionally, Jing et al. (2021) demonstrated how residual learning can improve ECG heartbeat classification. 1D ResNet can work in the middle stage between the increased computational complexity of CNN-recurrent and the simple use of a traditional CNN.

This study emphasizes assessing a customized 1d ResNet architecture for balanced categorization performance instead of just duplicating the highest accuracy from literature. Then the top-performing model will be identified by comparing three models and hyperparameter tuning will be applied on it. Finally, models evaluated with the original models by using multiple metrics like accuracy, precision, F1-score, recall, ROC-AUC and confusion matrix.

To conclude, this study connects previous works on CNN, CNN-BiLSTM, residual networks and attention-based models for classifying ECG heartbeats so that here it can compare these models and can optimize the best performing architecture. Also, it aims to determine the best-performed architecture by evaluating the consistency of improvements in different heartbeat classes and the impact of 1D ResNet model hyperparameter tuning on the generalization.

3.6 Selected Algorithm Summary

Finally, three architectures were selected for this study based on the reviewed literature and it provides complementary techniques for classifying ECG heartbeat problems. **Table 2** shows the summary of the selected three models.

No.	Architecture	What it learns	Reason of Selecting
1	1D CNN	Local ECG morphology.	Strong and computationally efficient baseline.
2	CNN-BiLSTM	Local morphology and temporal dependencies.	Tests whether recurrent temporal modelling improves performance.
3	1D ResNet	Deep hierarchical convolutional representations.	Tests deeper residual learning and provides the strongest literature support.
4	Tuned-1D ResNet	Optimised residual representations.	Determines whether hyperparameter optimisation further improves generalisation.

Table 2: Summary of the selected three models.

Because of performing as best model, 1D ResNet is chosen as the main model for hyperparameter tuning and its performance will be assessed using peer models from 1D CNN and CNN-BiLSTM. Here, it won't presume that ResNet is inherently better, it will be experimentally proven with comparison and further optimization. So, this selection of models maintains the project's experimental contribution while also being in line with the literature.

4. Discussion of Methods

4.1 Research Methodology

The goal of this study is to develop an experimental deep learning framework for investigating automated ECG heartbeat classification. Its main purpose is making the model capable of classifying the signals into five categories: Normal (N), Supraventricular (S), Ventricular (V), Fusion (F) and Unknow (Q). It is designed in a systematic way so that it will be easy to compare between different deep learning methods rather than relying on a single one.

In this study three models: 1D CNN, CNN-BiLSTM and 1D ResNet were used for ECG data classification. Here 1D CNN is used because it is efficient in finding the local pattern. CNN-BiLSTM is a combine framework of convolutional features and bidirectional temporal modeling and it helps to improve classification accuracy. And 1D ResNet is implemented using residual connections for facilitating deeper learning while maintaining steady gradient flow. That proves both CNN-BiLSTM and residual architectures are good for classifying ECG heartbeats.

Experimental process involves examining and preparing the ECG dataset, converting cardia signals into numerical data for 1d deep learning and splitting the dataset. For evaluating the performance of three models that are trained under comparable circumstances different types of metrics like: accuracy, precision, recall, F1-score, and ROC-AUC are used. And then top performed model 1D ResNet is used for additional hyperparameter tuning. The flow chart given in **Figure 4** represents the workflow of this project.

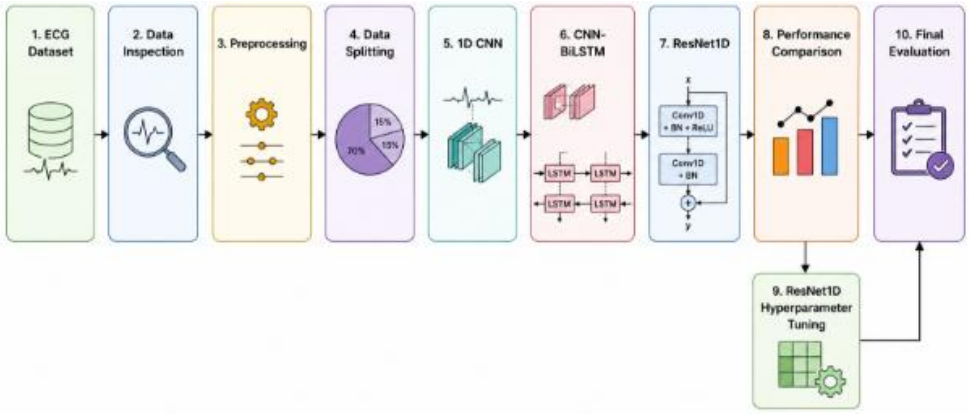


Figure 4: Workflow of the project.

This methodology ensures a systematic comparison of three deep learning models to representation learning and helps to investigate the hyperparameter optimization separately from the effect of architectural design.

4.2 Description of the Dataset

Here ECG Heartbeat Categorization Dataset from Kaggle is selected for this study which is created based on the MIT-BIH arrhythmia database. For researching automated ECG and heartbeat classification this database is very popular and it contains classified heartbeat segments which show different types of cardiac activity. The dataset contains 5 categories heartbeat shown in **Table 3**:

Class	Description
N	Normal heartbeat
S	Supraventricular ectopic heartbeat
V	Ventricular ectopic heartbeat
F	Fusion heartbeat
Q	Unknown heartbeat

Table 3: 5 categories heartbeat in the dataset.

This dataset is suitable for this study as it contains realistic different-class ECG classification problem instead of a simple binary classification problem. Also, the dataset shows a substantial class imbalance where normal pulse is much larger portion of the samples than some of the abnormal heartbeat categories. This offers an opportunity to examine the deep learning model's ability to acquire significant illustrations for both majority and minority classes. The details of the dataset are given below:

- Dataset: ECG Heartbeat Categorization Dataset
- Source: Kaggle
- Dataset link: https://www.kaggle.com/datasets/shayanfazeli/heartbeat

As the dataset is based on MIT-BIH Arrhythmia Database which is consistent with previous studies that's why it was selected. For example, Jing et al. (2021) Used MIT-BIH Arrhythmia

Database for implementing ResNet-18 architecture while khan et al. (2023) used the same dataset and implemented 1D convolutional deep residual network for classifying ECG heartbeat.

4.3 Dataset Preprocessing

The most crucial stage of implementing a deep-learning model is pre-processing because these models are sensitive to dimensionality, scale and consistency of the input data. First initial inspections were carried done which include identifying input structure, class labels, missing values and class distribution. Then the ECG data were converted to 1D numerical sequences which enables direct processing with 1D convolutional and recurrent layers.

4.3.1 Input Representation

ECG heartbeat signals are fixed length 1-dimensional signals and that's why it requires the model to learn patterns with the temporal dimensional. The given **equation 4** is general form of the input which is used for the deep learning models.

$$X \in R^{N \times T \times C} \quad \text{.....(4)}$$

Where, N = number of heartbeat samples.

T = number of time steps per heartbeat.

C = number of input channels.

The dataset of this study contains each heartbeat which is represented by a single ECG signal channel that enables the model to use the 1D convolutional operations.

4.3.2 Normalization

Then ECG data are normalized so that it can be more consistent numerical range and improve optimization before model building. This also can reduce the effect variations in signal amplitude and let the deep learning networks to focus on waveform features more effectively.

This method is relevant to deep leaning networks because it can scale the inputs appropriately which can improve gradient-based optimization and lessens instability during training.

4.3.3 Label Encoding

The classified heartbeat labels are converted to numerical expressions for supervised deep learning by using a multi-class encoding technique. A likelihood distribution among the heartbeat classes is produced using the softmax activation function and finally five neurons were set up the final output layer of each classification. The mathematical expression of encoding is shown in **equation 5.**

$$P(y = k|x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{.....(5)}$$

Where, k = 5 which indicates the number of heartbeat classes.

The class with the highest predicted likelihood produces the expected class.

4.4 Deep Leaning Models

To fulfil the requirement of comparing deep learning architectures of this study, three models were implemented. Each model showcases different approach to learning ECG representations.

4.4.1 Baseline 1D CNN

For the basis model of the project is 1D CNN. Its convolutional layers apply learnable filters in the temporal ECG sequence and these filters can identify local waveform features like peaks, amplitude, morphological patterns and valleys etc automatically. The structure of the 1D CNN is shown in **Figure 5**.

This architecture provides a suitable baseline as it can train ECG morphology without extracting features manually. Though convolutional CNNs can only depend on local

receptive class that might limit their capability to capture longer-range temporal dependencies.

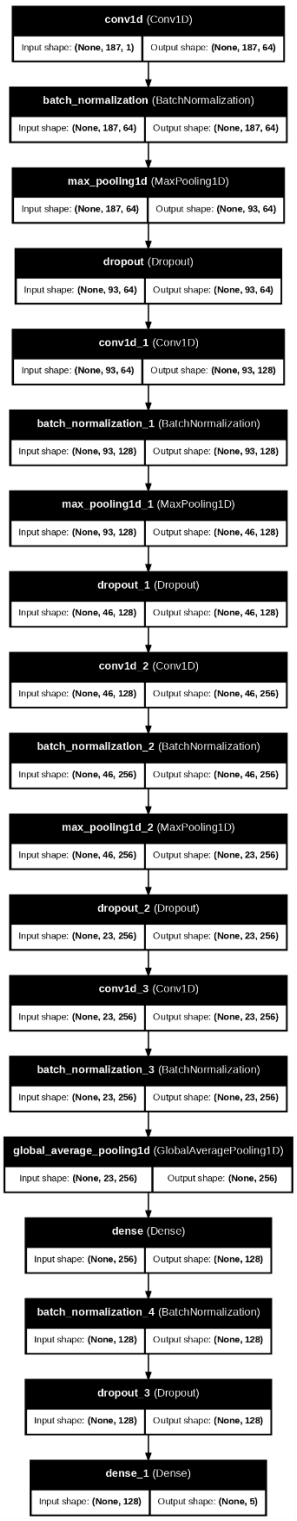


Figure 5: Structure of 1D-CNN

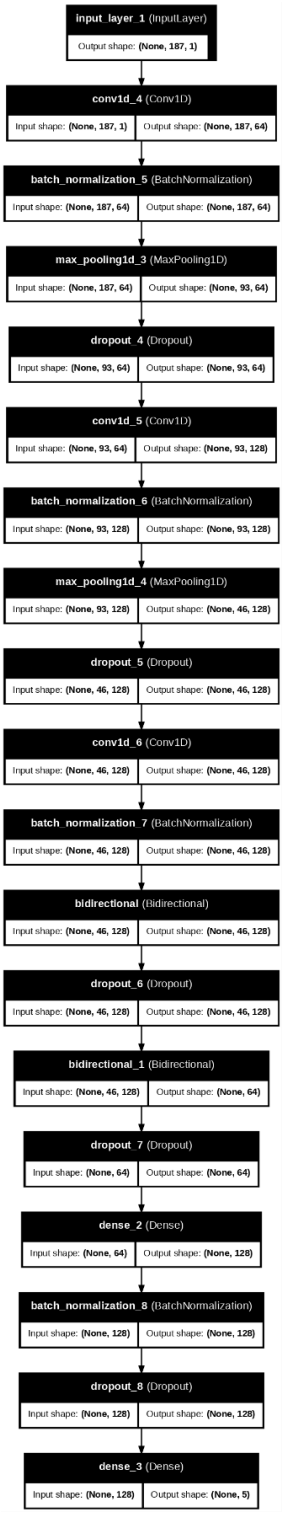


Figure 6: Structure of CNN-BiLSTM.

4.4.2 CNN-BiLSTM

The architecture combines a BiLSTM network with CNN because of classifying ECG signals. Here CNN extracts local features. And BiLSTM uses the temporal associations and local morphological information that includes in ECG data to process these characteristics sequentially in both directions. This architecture was supporting my previous research work by Hassan et al. (2022) where they used CNN-BiLSTM model and achieved 98% accuracy for five-class ECG heartbeat classification. By adding recurrent layers in CNN results in more trainable parameters and computing requirements compared to basic CNN. The structure of the CNN-BiLSTM is shown in **Figure 6.**

4.4.3 1D ResNet

The final architecture was selected 1D ResNet, it applies residual learning to 1 dimension ECG signals which allows layers to focus on learning residual mappings by using shortcut connections that enable information gradient flow. The architecture of this model contains input layer, several residual blocks, dense layer and a softmax classification. In a study of Jing et al. (2021) showed efficiency of ResNet in ECG classification and achieved accuracy of 96.50% while Khan et al. (2023) achieved 98.63% accuracy by using 1D convolutional residual network. The structure of the 1D ResNet is shown in **Figure 7.**

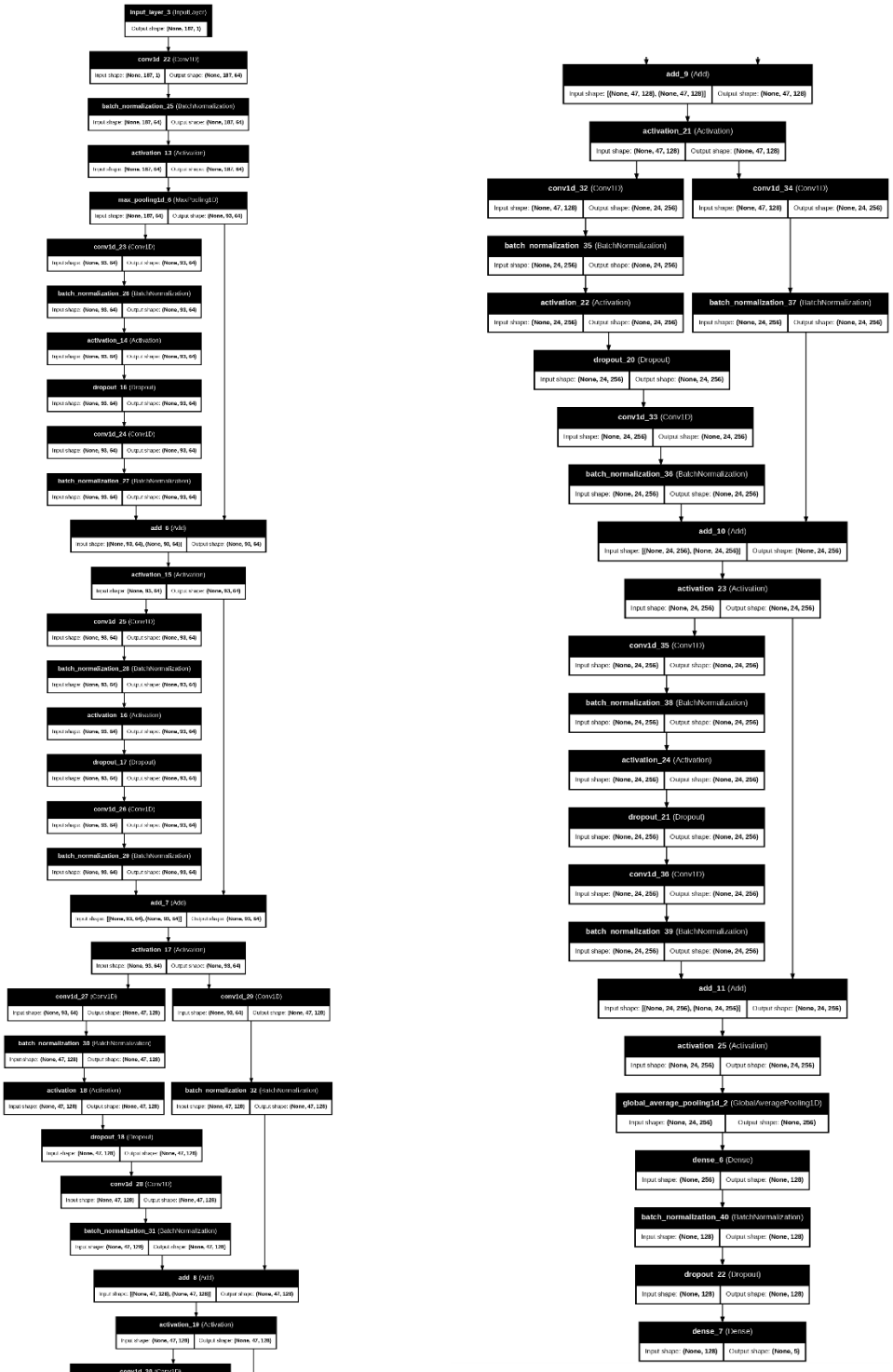


Figure 7: Structure of 1D-ResNet.

4.5 Model Training

All three models were trained by using supervised multi-class classifiers. First the five neurons in the last output layer means the five kinds of heartbeats. Then the model weights were improved by gradient-based optimization using Adam optimizer. For training large neural networks, Adam works better than any other optimizer because it combines momentum-based gradient estimations with tuned learning rates. The training purpose for multi-class classification is predicted on categorical cross-entropy. The **equation 6** shows the mathematical term of it.

$$L = -\sum_{i=1}^{K} y_i \log(\hat{y}_i) \quad \text{.....(6)}$$

Where, K = number of classes.

y_i = true class labels.

$\dot{Y}_i$ = predicted probability for class labels i.

Model performance is tracked during training by using both training and validation metrics which made it possible to analyze how the models work and support in spotting possible overfitting. The training procedure can include early stopping or learning rate control when it's needed. All these strategies can improve adaptability and avoid redundant training if validation performance stops improving.

4.6 Hyperparameter Tuning of 1D ResNet

After implementing all three models and initial comparison, the top performed architecture is optimized further by using hyperparameter tuning. In this framework, 1D ResNet performed best among the three models in the experimental study, so it was selected for this stage.

This step checks all the important parameters which can influence a model performance which includes: 1) learning rate, 2) residual block configuration, 3) number of filters, 4) dropout rate, 5) batch size, 6) number of training epochs and optimizer related parameters. Hyperparameter tuning is different from model training because the model parameters are learned automatically during training while hyperparameters are set up before or during optimization process.

The main purpose of tuning is not only increasing the training accuracy but also identifying the best configuration which will provide stronger validation performance and generalization. Lastly the tuned 1D ResNet model performance is evaluated on the held-out test set and compared with the original three models.

So, at the end of this framework, it produces total 4 models:

- 1D CNN.
- CNN-BiLSTM.
- 1D ResNet.
- Tuned 1D ResNet.

This assessment supports this study's objective directly as it is determining whether a deeper residual model can provide an advantage and whether systematic hyperparameter optimization can improve performance further.

4.7 Evaluation Metrics

For evaluating ECG classification problem accuracy is not sufficient as the dataset contains imbalanced heartbeat classes. That's why this study used several evaluation metrics.

4.7.1 Accuracy

The proportion of classifying samples correctly is known as accuracy. The function is shown in **equation 7.**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \qquad \text{.....(7)}$$

Accuracy shows the ratio of all heartbeats samples which are assigned to the correct category for multi-class classification. It might be useful as an overall measure but sometimes it might mislead when one class dominates the dataset.

4.7.2 Precision

Precision can measure the number of samples predicted for a particular class that actually belong to that class. The function is shown in **equation 8.**

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad \text{.....(8)}$$

A maximum value of precision means that the model produces less false positive predictions relatively.

4.7.3 Recall

Recall or sensitivity estimates the ratio of actual positive data which is identified correctly. The function is shown in **equation 9.**

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{.....(9)}$$

For medical filed classification, recall is more important as failing to be detecting an abnormal heartbeat class can be more harmful than incorrectly classifying a normal heartbeat as abnormal.

4.7.4 F1-Score

F1 score is combination of precision and recall via their harmonic mean value. The function is shown in **equation 10.**

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{.....(10)}$$

F1 score is better than others in this project as it can give equal importance to each pulse class rather than allowing majority class to dominate the metric.

4.7.5 ROC AUC

Here, ROC curve is used for analyzing the trade-off between true-positive and true-negative rates while AUC provides overall measures of discriminative ability. ROC-AUC can be

estimated by using one vs others strategy for a multi-class problem. AUC value near 1 means stronger discrimination between classes.

4.7.6 Confusion Matrix

For class-level analysis of the predictions, a confusion matrix is used. The rows indicate the actual classes, and the columns represent the predicted classes. It is important for this study because it allows the performance of the model to be investigated on minority classes rather than relying on accuracy only.

4.8 Model Comparison and Selection

By evaluating the models under a consistent experimental framework, it can ensure that the comparisons are meaningful. The 1D CNN builds the performance of conventional convolutional feature extraction. Then the CNN-BiLSTM model can investigate if explicit temporal modeling improves categorizations. Lastly, 1D ResNet assesses if deeper residual feature learning provides a benefit over alternative techniques.

The comparison is done by using metrics like accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix and training/validation curves as achieving higher accuracy may not provide better recognition of minor heartbeat classes properly. 1D ResNet was selected for further hyperparameter optimization based on the experimental outputs and compared against all three initial models. This explains a clear vision of both architectural differences and the effect of hyperparameter optimization.

Here, the methodology points out two key questions: 1) the performance of the best deep learning model classifying ECG heartbeat and 2) whether hyperparameter optimization can improve its performance and adaptability.

This architecture provides controlled experimental comparison within the dataset while keeping relevant with previous research demonstrating the effectiveness of convolutional, recurrent and residual architecture for ECG Classification (Hassan et al., 2022, Jing et al., 2021, Khan et al., 2023).

4.9 Summary

The procedure of this study satisfies the requirement to develop, compare, tune, validate and critically assess several deep learning systems while providing an organized and repeatable techniques for classifying ECG heartbeat. In addition, it makes clear connection between the experimental models investigated in this study and the reviewed literature used here.

5. Model Implementation

First, we need to import all the required libraries and GPU for developing the project. Here we used Google Colab to implement this project and **Figure 8** shows the required libraries we imported and the GPU we need for it.

```
!pip install -q kagglehub
```

```
import os
import gc
import random
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers, models

from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (classification_report,confusion_matrix, accuracy_score,
                             precision_score,recall_score, f1_score)

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

```
TensorFlow: 2.20.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Figure 8: Code and Output of importing libraries and GPU check in Google Colab.

5.1 Data Preprocessing and preparation

Here the implementation stages start with the preparation of ECG heartbeat dataset for building deep learning framework. The ECG Heartbeat Categorization Dataset was collected from Kaggle and it was downloaded directly in Google Colab Environment. The screenshot of the code and output are shown in **Figure 9**.

```
import kagglehub

dataset_path = kagglehub.dataset_download(
    "shayanfazeli/heartbeat"
)

print("Dataset path:")
print(dataset_path)

print("\nFiles:")
print(os.listdir(dataset_path))
```

```
Using Colab cache for faster access to the 'heartbeat' dataset.
Dataset path:
/kaggle/input/heartbeat

Files:
['ptbdb_abnormal.csv', 'ptbdb_normal.csv', 'mitbih_test.csv', 'mitbih_train.csv']
```

Figure 9: Code and Output of downloading dataset in Google Colab.

After downloading the files named mitbih_train.csv, mitbih_test.csv, ptbdb_abnormal.csv, and ptbdb_normal.csv, the MIT-BIH training and testing files were selected as they correspond to the five types of heartbeat classification problem examined in this project. The screenshot below in **Figure 10** shows the code and output of checking shapes in the dataset.

```
train_file = os.path.join(dataset_path, "mitbih_train.csv")
test_file = os.path.join(dataset_path, "mitbih_test.csv")
train_df = pd.read_csv(train_file, header=None)
test_df = pd.read_csv(test_file, header=None)

print("Training shape:", train_df.shape)
print("Testing shape:", test_df.shape)
```

```
Training shape: (87554, 188)
Testing shape: (21892, 188)
```

Figure 10: Code and Output of checking the shape of required dataset.

So, the output shows that the MIT-BIH training dataset has 87,554 data and the independent test dataset has 21,892 data with each data containing 187 ECG channels and one class label. Then according to **Figure 11**, for checking all the columns type and checking if there is any missing, the dataset was loaded into Pandas data frames.

```
print("First 5 rows:")
display(train_df.head())

print("\nData types:")
print(train_df.dtypes.value_counts())

print("\nMissing values:")
print("Train:", train_df.isnull().sum().sum())
print("Test :", test_df.isnull().sum().sum())
```

	0	1	2	3	4	5	6	7	8	9	...	178	179	180	181	182	183	184	185	186	187
0	0.977941	0.926471	0.681373	0.245098	0.154412	0.191176	0.151961	0.085784	0.058824	0.049020	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.960114	0.863248	0.461538	0.196581	0.094017	0.125356	0.099715	0.088319	0.074074	0.082621	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	1.000000	0.659459	0.186486	0.070270	0.070270	0.059459	0.056757	0.043243	0.054054	0.045946	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.925414	0.665746	0.541436	0.276243	0.196133	0.077348	0.071823	0.060773	0.066298	0.058011	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.967136	1.000000	0.830986	0.586854	0.356808	0.248826	0.145540	0.089202	0.117371	0.150235	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 188 columns

```
Data types:
float64    188
Name: count, dtype: int64

Missing values:
Train: 0
Test : 0
```

Figure 11: Code and Output of checking data types and missing values.

So, from the output, it's clear that 188 columns contain data and their types are numerical. And it also shows that there aren't any missing values in both training and testing datasets. Then in **Figure 12**, the code was written to separate the feature matrix and target vector. The output

shows training data contains 87,554 heartbeat data with 187 signals per data while independent test set contains 21,892 data.

```
X_train_full = train_df.iloc[:, :-1].values.astype("float32")
y_train_full = train_df.iloc[:, -1].values.astype("int32")

X_test = test_df.iloc[:, :-1].values.astype("float32")
y_test = test_df.iloc[:, -1].values.astype("int32")

print("X_train_full:", X_train_full.shape)
print("y_train_full:", y_train_full.shape)

print("X_test:", X_test.shape)
print("y_test:", y_test.shape)
```

```
X_train_full: (87554, 187)
y_train_full: (87554,)
X_test: (21892, 187)
y_test: (21892,)
```

Figure 12: Code and Output after separating the feature matrix and target vector.

Then the total heartbeat classes were checked and encoded like **Figure 13**. These classes were extracted as integer so that sparse categorical cross-entropy can be implemented during model training.

```
class_names = { 0: "Normal (N)",
                1: "Supraventricular (S)",
                2: "Ventricular (V)",
                3: "Fusion (F)",
                4: "Unknown (Q)" }

num_classes = 5

print("Classes:")

for i in range(num_classes):
    print(i, "=", class_names[i])
```

```
Classes:
0 = Normal (N)
1 = Supraventricular (S)
2 = Ventricular (V)
3 = Fusion (F)
4 = Unknown (Q)
```

Figure 13: Code and Output for checking category classes and encoding.

Then screen short in **Figure 14** shows the code of checking class distribution. Where the output shows that the dataset is strongly imbalanced. Only the Normal class is taking 82.77% of the training data where Fusion is taking 0.73%, Supraventricular and Ventricular taking 2.54% and 6.61% respectively and lastly unknown occupied only 7.35% of the training dataset. So, the dataset because the class distribution is highly skewed as the normal pulses are substantially exceeding the minor classes.

```
classes, counts = np.unique(y_train_full, return_counts=True)

distribution_df = pd.DataFrame({
    "Class": classes,
    "Name": [class_names[c] for c in classes],
    "Count": counts,
    "Percentage": counts / len(y_train_full) * 100
})

display(distribution_df)
```

	Class	Name	Count	Percentage
0	0	Normal (N)	72471	82.772917
1	1	Supraventricular (S)	2223	2.539005
2	2	Ventricular (V)	5788	6.610777
3	3	Fusion (F)	641	0.732120
4	4	Unknown (Q)	6431	7.345181

```
plt.figure(figsize=(10, 6))

sns.barplot(data=distribution_df, x="Name", y="Count")

plt.title("ECG Heartbeat Class Distribution", fontsize=16)

plt.xlabel("Heartbeat Class")
plt.ylabel("Number of Samples")

plt.xticks(rotation=20)

plt.tight_layout()

plt.show()
```

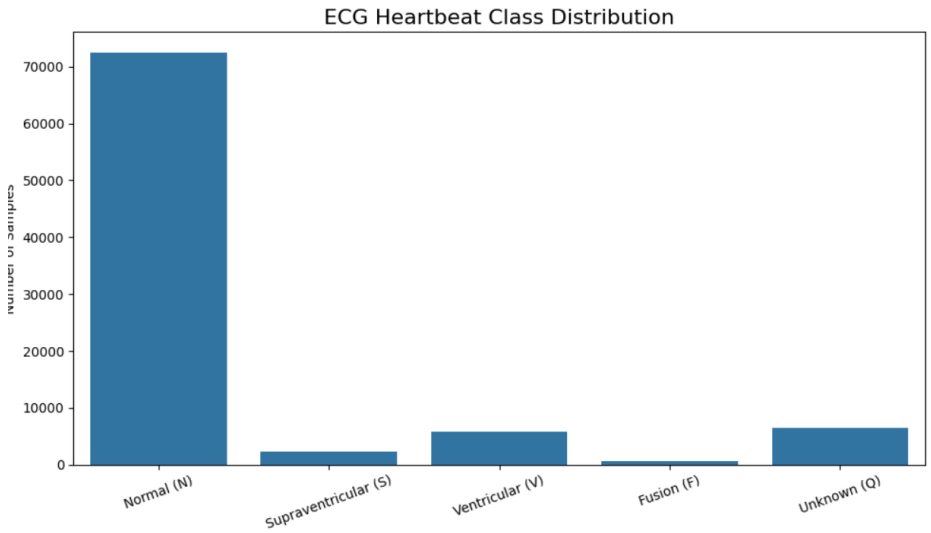


Figure 14: Code and Output for checking class distribution.

This finding is an important fact as a model can not achieve better accuracy by favoring the majority Normal class while performing poorly on clinically critical minority classes. Hassan et al. (2022) found this similar identification of class imbalance as an important methodological issue.

5.2 Handling Class Imbalance

From the output of class distribution, we found that the classes highly imbalanced and it should be handled before implementing any model. So, to reduce the effect of it, class weights were assigned into the learning process.

First, we calculated the initial full-balanced weights using the code given in **Figure 15**. This project didn't apply conventional balanced class weights directly. According to the code in the

implementation, we applied squared root of the calculated weights. This technique is suitable for the dataset because the class distribution was highly skewed. And it provides less aggressive reweighting strategy while giving proper importance to the minority class. As like **Figure 15** output, same moderate weights were used in the 1D CNN, CNN-BiLSTM and 1D ResNet model during training.

```
original_class_weights = compute_class_weight(class_weight="balanced",
                                              classes=np.unique(y_train),
                                              y=y_train)

class_weights = {int(cls): float(np.sqrt(weight))
                 for cls, weight in zip(
                   np.unique(y_train),
                   original_class_weights)
                }

print("Moderated class weights:")

for cls, weight in class_weights.items():
    print(f"{cls} - {class_names[cls]}: {weight:.4f}")
```

```
Moderated class weights:
0 - Normal (N): 0.4916
1 - Supraventricular (S): 2.8069
2 - Ventricular (V): 1.7394
3 - Fusion (F): 5.2256
4 - Unknown (Q): 1.6501
```

Figure 15: Code and Output for handling class imbalance.

5.3 Training/Test Split

Same training methods were used for the three models for ensuring a fair comparison. The MIT-BIH test set was preserved for separate analysis after original dataset was split into 80:20% for training and testing respectively shown in **Figure 16**.

```
X_train, X_val, y_train, y_val = train_test_split(X_train_full,y_train_full,test_size=0.20,
                                                  random_state=SEED, stratify=y_train_full)

print("Training:", X_train.shape)
print("Validation:", X_val.shape)
print("Testing:", X_test.shape)
```

```
Training: (70043, 187)
Validation: (17511, 187)
Testing: (21892, 187)
```

Figure 16: Code and Output of dataset splitting.

This produced three datasets: a test dataset for final evaluation for preventing optimize bias, a validation dataset for monitoring performance and lastly training dataset for model learning. Also, for making it appropriate for the model those were converted tensorflow datasets for

modeling shown in **Figure 17**. Each model was trained for 30 epochs using early stopping and learning rate reduction based on the validation results so that it allows for flexibility and avoid over-training.

```
BATCH_SIZE = 256
train_ds = (tf.data.Datasetfrom_tensor_slices((X_train, y_train))shuffle(20000,seed=SEED).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE))
val_ds = (tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE))
test_ds = (tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE))

print("TensorFlow datasets created.")
```

```
TensorFlow datasets created.
```

Figure 17: Code and Output of converting dataset to tensorflow for modeling.

5.4 1D CNN Model (Baseline)

As for the baseline model, this project implements 1D-CNN model. This model was selected because ECG heartbeat data contains 1-dimensional temporal signals. And convolutional filters can learn local morphological features directly from the signal.

The code for the 1D CNN model is given in **Figure 17**. In subsection **4.4.1** and **Figure 5**, the architecture of 1D CNN is shown. The input shape was defined as (187, 1), here 187 means the number of signal channels and 1 means the class of ECG signal. The structure has many convolutional blocks, first one uses 64 filters with size of kernel 7 followed by batch normalization, max polling and dropout. Next blocks increased the number of filters to 128 and 256 respectively while reducing the kernel size to 5 and 3. Then final convolutional stage was done with global average pooling and dense layer.

Here we used global average pooling instead of a large flattening layer because it reduces the number of parameters in the final classifier and provides strong visualization of the learned convolutional characteristics. This model used Adam optimizer with initial learning rate of 0.001 and as for the loss function, it used sparse categorical cross entropy.

```
def build_cnn(filters1=64,filters2=128,filters3=256,dropout=0.30,learning_rate=0.001):

    model = models.Sequential(name="Baseline_1D_CNN")
    model.add(layers.Input(shape=(187, 1)))

    # Block 1
    model.add(layers.Conv1D(filters1,kernel_size=7,padding="same",activation="relu"))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling1D(pool_size=2))
    model.add(layers.Dropout(dropout))

    # Block 2
    model.add(layers.Conv1D(filters2,kernel_size=5,padding="same",activation="relu"))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling1D(pool_size=2))
    model.add(layers.Dropout(dropout))

    # Block 3
    model.add(layers.Conv1D(filters3,kernel_size=3,padding="same",activation="relu"))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling1D(pool_size=2))
    model.add(layers.Dropout(dropout))

    # Block 4
    model.add(layers.Conv1D(filters3,kernel_size=3,padding="same",activation="relu"))
    model.add(layers.BatchNormalization())

# Global feature extraction
model.add(layers.GlobalAveragePooling1D())

# Dense classifier
model.add(layers.Dense(128,activation="relu"))
model.add(layers.BatchNormalization())
model.add(layers.Dropout(0.40))

model.add(layers.Dense(num_classes,activation="softmax"))

model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])

return model

cnn = build_cnn()

cnn.summary()
```

Figure 17: Code of 1D CNN Model.

Training 1D CNN Model

According to the Code and output in **Figure 18**, the CNN model was trained to 30 epochs with moderate class weights. Then callbacks were used to track validation performance and for tuning the learning rate when improvement slowed. Both training and validation performance showed good improvement like in last training epoch, it reached almost 98% validation accuracy with reduced learning rate when validation loss stopped improving.

According to **Figure 19**, the model achieved 98.30% accuracy in test accuracy. And all of the evaluation metrics results are shown in **Figure 20** as proof that the result ensures that this model is strong baseline for comparing with more complex models.

```
cnn_callbacks = create_callbacks("cnn")

history_cnn = cnn.fit(train_ds,validation_data=val_ds,epochs=30, class_weight=class_weights,
                      callbacks=cnn_callbacks,verbose=1)
```

```
Epoch 1/30
274/274 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - accuracy: 0.5866 - loss: 0.9167
Epoch 1: val_loss improved from None to 9.88983, saving model to /content/best_cnn.keras

Epoch 1: finished saving model to /content/best_cnn.keras
274/274 ━━━━━━━━━━━━━━━━━━━━ 29s 49ms/step - accuracy: 0.7593 - loss: 0.6085 - val_accuracy: 0.0661 - val_loss: 9.8898 - learning_rate: 0.0010
Epoch 2/30
273/274 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9190 - loss: 0.3118
Epoch 2: val_loss improved from 9.88983 to 1.64149, saving model to /content/best_cnn.keras

Epoch 2: finished saving model to /content/best_cnn.keras
274/274 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - accuracy: 0.9269 - loss: 0.2877 - val_accuracy: 0.1685 - val_loss: 1.6415 - learning_rate: 0.0010

Epoch 29: ReduceLROnPlateau reducing learning rate to 7.812500371073838e-06.
274/274 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - accuracy: 0.9820 - loss: 0.0676 - val_accuracy: 0.9834 - val_loss: 0.0630 - learning_rate: 1.5625e-05
Epoch 30/30
272/274 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9822 - loss: 0.0656
Epoch 30: val_loss did not improve from 0.06150
274/274 ━━━━━━━━━━━━━━━━━━━━ 5s 17ms/step - accuracy: 0.9826 - loss: 0.0639 - val_accuracy: 0.9829 - val_loss: 0.0633 - learning_rate: 7.8125e-06
Restoring model weights from the end of the best epoch: 25.
```

Figure 18: Code of Training 1D CNN Model.

```
cnn_test_loss, cnn_test_accuracy = cnn.evaluate(test_ds,verbose=1)

print(f"1D CNN Test Accuracy: "f"{cnn_test_accuracy * 100:.2f}%")
```

```
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.9830 - loss: 0.0616
1D CNN Test Accuracy: 98.30%
```

Figure 19: Code and output of test accuracy of 1D CNN.

```
cnn_report = classification_report(y_test,cnn_predictions,
                                   target_names=[class_names[i]
                                                 for i in range(num_classes)],digits=4,zero_division=0)

print(cnn_report)
```

```
                    precision    recall  f1-score   support

         Normal (N)     0.9944    0.9879    0.9911     18118
Supraventricular (S)     0.7751    0.8741    0.8216       556
     Ventricular (V)     0.9667    0.9620    0.9643      1448
          Fusion (F)     0.6938    0.8951    0.7817       162
         Unknown (Q)     0.9882    0.9932    0.9907      1608

            accuracy                         0.9830     21892
           macro avg     0.8836    0.9424    0.9099     21892
        weighted avg     0.9843    0.9830    0.9835     21892
```

Figure 20: Code and output of results of 1D CNN.

5.5 CNN-BiLSTM Model

The second model was a hybrid model which is CNN-BiLSTM, it was chosen as ECG signals has both morphological local and temporal dependencies. Here, CNN will extract the local characteristics and BiLSTM will model sequential data in both forward and backward directions.

Like in subsection **4.4.2** and **Figure 6,** the structure starts with the CNN with 64 filter convolutional layer with kernel size of 7 followed by batch normalization, max pooling and dropout. More convolutional layers increased the feature representation to 128 channels.

Then the features extracted from CNN were passed to Two bidirectional LSTM layers. Fist layer contains 64 LSTM units and returns features, then it passes to second layer consisting of 32 units for continuing the temporal representation for classification. The final classification stage contains a dense layer with 128 neurons, drop out, batch normalization and a five neuron softmax output layer. The code for this is shown in **Figure 21**.

```
def build_cnn_bilstm(filters=64,lstm_units=64,dropout=0.30,learning_rate=0.001):

    inputs = layers.Input(shape=(187, 1))

    # ==========================
    # CNN FEATURE EXTRACTION
    # ==========================
    x = layers.Conv1D(filters=filters,kernel_size=7,padding="same",activation="relu")(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling1D(pool_size=2)(x)
    x = layers.Dropout(dropout)(x)
    x = layers.Conv1D(filters=filters * 2,kernel_size=5,padding="same",activation="relu")(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling1D(pool_size=2)(x)
    x = layers.Dropout(dropout)(x)
    x = layers.Conv1D(filters=filters * 2,kernel_size=3,padding="same",activation="relu")(x)
    x = layers.BatchNormalization()(x)

    # ==========================
    # TEMPORAL LEARNING
    # ==========================
    x = layers.Bidirectional(layers.LSTM(lstm_units,return_sequences=True))(x)
    x = layers.Dropout(dropout)(x)
    x = layers.Bidirectional(layers.LSTM(lstm_units // 2,return_sequences=False))(x)
    x = layers.Dropout(0.40)(x)

# ==========================
# CLASSIFIER
# ==========================
x = layers.Dense(128,activation="relu")(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.40)(x)
outputs = layers.Dense(num_classes,activation="softmax")(x)
model = models.Model(inputs,outputs,name="CNN_BiLSTM")

model.compile(optimizer=tf.keras.optimizers.Adam(
              learning_rate=learning_rate),loss="sparse_categorical_crossentropy",metrics=["accuracy"])

return model

cnn_bilstm = build_cnn_bilstm()
cnn_bilstm.summary()
```

Figure 21: Code of CNN-BiLSTM Model.

Training CNN-BiLSTM

The code for this part is shown in **Figure 22**. First CNN-BiLSTM showed unstable validation results but later it improved slowly after few epochs. It improved from 6.32% to 95.23% by the third epoch and later epoch it was around 98.50%. Here the learning rate reduction approach was activated consequently when validation loss stopped improving. According to **Figure 23** The model achieved a test accuracy of 98.40%, then all of the evaluation metrics results are shown in **Figure 24** as a proof. Although CNN-BiLSTM showed strong performance, it did not outperformed CNN That much, the difference is only .10%. So, this is an important fact which suggests that hybrid models do not automictically produce superior results for this dataset and pre-processing objectives.

```
cnn_bilstm_callbacks = create_callbacks("cnn_bilstm")

history_cnn_bilstm = cnn_bilstm.fit(train_ds,validation_data=val_ds,epochs=30,
                                    class_weight=class_weights,callbacks=cnn_bilstm_callbacks,verbose=1)
```

```
Epoch 11: val_loss did not improve from 0.07319
274/274 ━━━━━━━━━━━━━━━━━━━━ 10s 37ms/step - accuracy: 0.9744 - loss: 0.1022 - val_accuracy: 0.9784 - val_loss: 0.0787 - learning_rate: 5.0000e-04
Epoch 12/30
274/274 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.9747 - loss: 0.0978
Epoch 12: val_loss did not improve from 0.07319

Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
274/274 ━━━━━━━━━━━━━━━━━━━━ 10s 36ms/step - accuracy: 0.9751 - loss: 0.0994 - val_accuracy: 0.9724 - val_loss: 0.0920 - learning_rate: 5.0000e-04
Epoch 13/30
274/274 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.9758 - loss: 0.0930
Epoch 13: val_loss improved from 0.07319 to 0.06585, saving model to /content/best_cnn_bilstm.keras

Epoch 20: val_loss did not improve from 0.05606
274/274 ━━━━━━━━━━━━━━━━━━━━ 10s 36ms/step - accuracy: 0.9834 - loss: 0.0629 - val_accuracy: 0.9827 - val_loss: 0.0626 - learning_rate: 6.2500e-05
Epoch 21/30
273/274 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.9830 - loss: 0.0633
Epoch 21: val_loss did not improve from 0.05606

Epoch 21: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.
274/274 ━━━━━━━━━━━━━━━━━━━━ 10s 36ms/step - accuracy: 0.9831 - loss: 0.0627 - val_accuracy: 0.9832 - val_loss: 0.0597 - learning_rate: 6.2500e-05
Epoch 21: early stopping
Restoring model weights from the end of the best epoch: 15.
```

Figure 22: Code of Training CNN-BiLSTM Model.

```
bilstm_test_loss, bilstm_test_accuracy = (cnn_bilstm.evaluate(test_ds,verbose=1))

print(f"CNN-BiLSTM Test Accuracy: " f"{bilstm_test_accuracy * 100:.2f}%")
```

```
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.9840 - loss: 0.0570
CNN-BiLSTM Test Accuracy: 98.40%
```

Figure 23: Code and output of test accuracy of CNN-BiLSTM.

```
print(classification_report(y_test,bilstm_predictions,
                            target_names=[class_names[i]
                                          for i in range(num_classes)],digits=4,zero_division=0))
```

```
                    precision    recall  f1-score   support

          Normal (N)     0.9917    0.9921    0.9919     18118
Supraventricular (S)     0.8032    0.8219    0.8124       556
     Ventricular (V)     0.9711    0.9510    0.9609      1448
           Fusion (F)     0.8070    0.8519    0.8288       162
         Unknown (Q)     0.9913    0.9925    0.9919      1608

            accuracy                         0.9840     21892
           macro avg     0.9129    0.9219    0.9172     21892
        weighted avg     0.9842    0.9840    0.9841     21892
```

Figure 24: Code and output of results of CNN-BiLSTM.

5.6 1D ResNet Model

The last model we used here is 1D ResNet as it is suitable for the features of ECG signals and in previous work of Khan et al. (2023) it had strong performance. Like in **subsection 4.4.3** and **Figure 7** architecture, and the code given in **Figure 25**, this model begins with initial 64 filter Conv1D layer with kernel size of 7 and attached by batch normalization, ReLU activation and max pooling. The three residual stages consist of 2 blocks with 64 filters, 128 filters and 256 filters respectively. And down-sampling was used in selected residual blocks which use stride of 2. Then the learned patterns were passed via global average pooling consisting of 128 neuron dense layers, batch normalization, dropout layer and a five class softmax output layer.Lastly, the model use Adam with 0.001 learning rate and sparse categorical cross-entropy.

The advantage of ResNet is that the residual connections let the network learn deeper patterns while giving direct way for gradients and data. And this is the reason why this model is suitable for checking deeper feature extraction improves heartbeat classification or not.

```
def residual_block(x,filters,kernel_size=5,stride=1,dropout=0.20):

    shortcut = x

    # First convolution
    x = layers.Conv1D(filters=filters,kernel_size=kernel_size,strides=stride,padding="same",use_bias=False)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(dropout)(x)

    # Second convolution
    x = layers.Conv1D(filters=filters,kernel_size=kernel_size,padding="same",use_bias=False)(x)
    x = layers.BatchNormalization()(x)

    # Match shortcut dimensions
    if (shortcut.shape[-1] != filters
        or stride != 1):

        shortcut = layers.Conv1D(filters=filters,kernel_size=1,strides=stride,padding="same",use_bias=False)(shortcut)
        shortcut = layers.BatchNormalization()(shortcut)

    # Residual addition
    x = layers.Add()([x,shortcut])x = layers.Activation("relu")(x)

    return x
```

```
  def build_resnet1d(dropout=0.20,learning_rate=0.001):

      inputs = layers.Input(shape=(187, 1))

      # Initial feature extraction
      x = layers.Conv1D(64,kernel_size=7,padding="same",use_bias=False)(inputs)
      x = layers.BatchNormalization()(x)
      x = layers.Activation("relu")(x)
      x = layers.MaxPooling1D(pool_size=2)(x)

      # ============================
      # RESIDUAL STAGE 1
      # ============================
      x = residual_block(x,filters=64,kernel_size=5,dropout=dropout)
      x = residual_block(x,filters=64,kernel_size=5,dropout=dropout)

      # ============================
      # RESIDUAL STAGE 2
      # ============================
      x = residual_block(x,filters=128,kernel_size=5,stride=2,dropout=dropout)
      x = residual_block(x,filters=128,kernel_size=5,dropout=dropout)

      # ============================
      # RESIDUAL STAGE 3
      # ============================
      x = residual_block(x,filters=256,kernel_size=3,stride=2,dropout=dropout)
      x = residual_block(x,filters=256,kernel_size=3,dropout=dropout)

      # ============================
      # CLASSIFIER
      # ============================
      x = layers.GlobalAveragePooling1D()(x)
      x = layers.Dense(128,activation="relu")(x)
      x = layers.BatchNormalization()(x)
      x = layers.Dropout(0.40)(x)
      outputs = layers.Dense(num_classes,activation="softmax")(x)

      model = models.Model(inputs,outputs,name="ResNet1D")
      model.compile(optimizer=tf.keras.optimizers.Adam(
                    learning_rate=learning_rate),loss="sparse_categorical_crossentropy",metrics=["accuracy"])
      return model

  resnet1d = build_resnet1d()
  resnet1d.summary()
```

Figure 25: Code of 1D ResNet Model.

Training 1D ResNet Model

In **Figure 26** the model showed rapid improvement during training. First epoch showed poor validation, but it improved consequently during subsequent epochs. It ended with validation accuracy of 98.86% with reduced learning rate. Then **Figure 27** Shows the test accuracy of 1D ResNet which is 98.72% which .32% and .42% higher than CNN-BiLSTM and 1D-CNN. Also, all of the evaluation metrics results are shown in **Figure 28** as proof that this model has the highest performance among the three initial architectures, and it means 1D ResNet is selected for the subsequent hyperparameter tuning stage.

```
resnet_callbacks = create_callbacks("resnet1d")
history_resnet = resnet1d.fit(train_ds,validation_data=val_ds,epochs=30,
                              class_weight=class_weights,callbacks=resnet_callbacks,verbose=1)
```

```
Epoch 1/30
274/274 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step - accuracy: 0.6941 - loss: 0.7207
Epoch 1: val_loss improved from None to 9.16147, saving model to /content/best_resnet1d.keras

Epoch 1: finished saving model to /content/best_resnet1d.keras
274/274 ━━━━━━━━━━━━━━━━━━━━ 48s 91ms/step - accuracy: 0.8492 - loss: 0.4446 - val_accuracy: 0.0702 - val_loss: 9.1615 - learning_rate: 0.0010
Epoch 2/30
273/274 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.9449 - loss: 0.2284
Epoch 2: val_loss improved from 9.16147 to 1.88448, saving model to /content/best_resnet1d.keras

    Epoch 28/30
    273/274 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.9946 - loss: 0.0158
    Epoch 28: val_loss did not improve from 0.04479
    274/274 ━━━━━━━━━━━━━━━━━━━━ 10s 37ms/step - accuracy: 0.9948 - loss: 0.0155 - val_accuracy: 0.9884 - val_loss: 0.0457 - learning_rate: 3.1250e-05
    Epoch 29/30
    273/274 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 0.9952 - loss: 0.0149
    Epoch 29: val_loss did not improve from 0.04479

    Epoch 29: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.
    274/274 ━━━━━━━━━━━━━━━━━━━━ 10s 37ms/step - accuracy: 0.9950 - loss: 0.0155 - val_accuracy: 0.9886 - val_loss: 0.0457 - learning_rate: 3.1250e-05
    Epoch 29: early stopping
    Restoring model weights from the end of the best epoch: 23.
```

Figure 26: Code of Training 1d ResNet Model.

```
resnet_test_loss, resnet_test_accuracy = (resnet1d.evaluate(test_ds,verbose=1))

print(f"ResNet1D Test Accuracy: "f"{resnet_test_accuracy * 100:.2f}%")
```

```
86/86 ━━━━━━━━━━━━━━━━━━━━ 2s 23ms/step - accuracy: 0.9872 - loss: 0.0514
ResNet1D Test Accuracy: 98.72%
```

Figure 27: Code and output of test accuracy of 1D ResNet.

```
print(classification_report(y_test,resnet_predictions,target_names=[class_names[i]
                                                                    for i in range(num_classes)],digits=4,zero_division=0))
```

```
                    precision    recall  f1-score   support

         Normal (N)     0.9932    0.9942    0.9937     18118
Supraventricular (S)     0.8767    0.8309    0.8532       556
     Ventricular (V)     0.9756    0.9662    0.9709      1448
          Fusion (F)     0.7688    0.8827    0.8218       162
         Unknown (Q)     0.9913    0.9919    0.9916      1608

            accuracy                         0.9872     21892
           macro avg     0.9211    0.9332    0.9262     21892
        weighted avg     0.9873    0.9872    0.9872     21892
```

Figure 28: Code and output of results of 1D ResNet.

5.7 Model Evaluation

As the dataset was detected as highly imbalanced that's why model performance was not evaluated with accuracy only. It included evaluation metrics like Precision, Recall, F1-Score, ROC-AUC and Confusion Matrix along with Accuracy as it is used in **Figure 20, 24 and 28**. The Code and Output for the initial three model in given in **Figure 29** for better understanding of the evaluation stage of this study. All of these metrics provide both overall evaluation and a class balanced assessment of performance.

```
results = pd.DataFrame([
    calculate_metrics(y_test,cnn_predictions,"1D CNN"),
    calculate_metrics(y_test,bilstm_predictions,"CNN-BiLSTM"),
    calculate_metrics(y_test,resnet_predictions,"ResNet1D")])

display(results.round(4))
```

	Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Weighted F1
0	1D CNN	0.9830	0.8836	0.9424	0.9099	0.9835
1	CNN-BiLSTM	0.9840	0.9129	0.9219	0.9172	0.9841
2	ResNet1D	0.9872	0.9211	0.9332	0.9262	0.9872

Figure 29: Code and output of results of three model evaluation.

The results of the initial models achieved very high accuracy according to the output in **Figure 29**. Though 1D ResNet achieved accuracy of 98.72% which is .32% and .42% higher than CNN-BiLSTM (98.40%) and 1D-CNN (98.30%). But it should not check this only like here 1D CNN has the strongest recall among three models while 1D ResNet has the strongest precision, f1- and score.

5.8 Architectural Modifications and Original Contributions

The model development of this study tried multiple customized design options for improving the performance of the ECG heartbeat dataset. Some of the main contributions are listed below:

a. Moderate Weighted Class: For handling dataset imbalance, a squared-root transformation of balance class weights effectively without excessively weighting minor classes.
b. Controlled Structure Comparison: With a unified experimental configuration evaluated three different deep learning models which enabled direct comparison of learning methods.

c. Dataset-specified designed 1D ResNet: for incorporating 1-dimensional convolutions and a five-class categorizer, customized the 1D ResNeT structure for a 187-point 1-dimensional ECG input.
d. Optimized 1D ResNet systemically: Conduct evaluation of six 1D ResNet model configurations and choose the best configuration with characterized by 128 filters, a kernel size of 7, dropout rate of 0.30 and learning rate of 0.0005 which will be discussed more in **Section 6**. It was used for building the Final tuned ResNet 1D.

5.9 Summary

Here a steady method of development is demonstrated. CNN proved itself as a robust convolutional baseline while CNN-BiLSTM examined if explicit bidirectional temporal modeling could provide more advantages. Lastly comparing three initial models 1D ResNet was chosen for optimization since it had the best performance. Details related to hyperparameter tuning and tuned model results will be discussed in Section 6.

6. Hyperparameter Tuning, Validation and Final Evaluation

6.1 Hyperparameter Tuning Strategy

Based on the initial comparison of three models in **Figure 29**, 1D ResNet achieved the best accuracy among three models and that's why it was selected as the final model for further optimization. The basic 1D ResNet model got 98.72% of accuracy compared with CNN-BiLSTM (98.40%) and 1D-CNN (98.30%). As it performed better, rather than tuning all models, this study selected the best model for the tuning stage.

The purpose of hyperparameter tuning is to make a model perform best by changing appropriate hyperparameters. So here tuning was done to determine whether appropriate changing in 1D ResNet configuration can enhance its overall performance. For 1D ResNet, three hyperparameters: convolutional filters, learning rate and dropout rate were investigated because these three hyperparameters can influence directly to the regularization, representational ability and optimization pattern of the network

Total 6 variations of configurations were evaluated for tuning. The configuration includes:

a. Number of Filters from 64 to 128.
b. Dropout rates are between 0.20 and 0.30.
c. Learning rates are between 0.0005 and 0.001.

The code of the configuration is given in **Figure 30**. Each configuration was trained using the same training and validation process so that the outputs can be evaluated and compared fairly.

```
tuning_configs = [
    {"base_filters": 32,"dropout": 0.20,"learning_rate": 0.001},
    {"base_filters": 64,"dropout": 0.20,"learning_rate": 0.001},
    {"base_filters": 64,"dropout": 0.30,"learning_rate": 0.001},
    {"base_filters": 64,"dropout": 0.30,"learning_rate": 0.0005},
    {"base_filters": 64,"dropout": 0.40,"learning_rate": 0.0005},
    {"base_filters": 128,"dropout": 0.30,"learning_rate": 0.0005}
]

print("Number of ResNet1D configurations:", len(tuning_configs))
```

```
Number of ResNet1D configurations: 6
```

Figure 29: Code and output of tuning configuration.

The validation performance was used to select the best configuration because this is more important than final test set as using test set continuously during tuning might introduce test-set information inside the model-selection procedure.

6.2 Tuning Results and Model Selection

Then six configurations were executed respectively and provide six different validation performance. The results of the six configurations are given in **Figure 31**.

```
tuning_results_df = pd.DataFrame(tuning_results)
tuning_results_df = tuning_results_df.sort_values(by="Macro F1",ascending=False).reset_index(drop=True)
display(tuning_results_df.round(4))
```

	Configuration	Base Filters	Dropout	Learning Rate	Validation Accuracy	Macro Precision	Macro Recall	Macro F1	Macro ROC-AUC	Epochs
0	6	128	0.3	0.0005	0.9842	0.9074	0.9171	0.9087	0.9959	13
1	4	64	0.3	0.0005	0.9797	0.8572	0.9303	0.8863	0.9950	14
2	1	32	0.2	0.0010	0.9724	0.8878	0.8646	0.8744	0.9903	6
3	2	64	0.2	0.0010	0.9721	0.9208	0.8457	0.8740	0.9912	7
4	3	64	0.3	0.0010	0.9682	0.8585	0.8526	0.8526	0.9878	6
5	5	64	0.4	0.0005	0.9592	0.7813	0.8992	0.8091	0.9891	9

Figure 31: Six configuration results of 1D ResNet.

From the output **Figure 31 and 32**, we see that configuration 6 performed the best among six configurations based on validation performance. It achieved 98.42% validation accuracy and 90.87% validation macro F1-score.

```
best_config = tuning_results_df.iloc[0]

print("=" * 70)
print("BEST TUNED RESNET1D CONFIGURATION")
print("=" * 70)
print(f"Base Filters   : {int(best_config['Base Filters'])}")
print(f"Dropout        : {best_config['Dropout']}")
print(f"Learning Rate  : {best_config['Learning Rate']}")
print("\nValidation Performance")
print("-" * 50)
print(f"Accuracy       : {best_config['Validation Accuracy']:.4f}")
print(f"Macro Precision: {best_config['Macro Precision']:.4f}")
print(f"Macro Recall   : {best_config['Macro Recall']:.4f}")
print(f"Macro F1       : {best_config['Macro F1']:.4f}")
print(f"Macro ROC-AUC  : {best_config['Macro ROC-AUC']:.4f}")
```

```
======================================================================
BEST TUNED RESNET1D CONFIGURATION
======================================================================
Base Filters   : 128
Dropout        : 0.3
Learning Rate  : 0.0005

Validation Performance
--------------------------------------------------
Accuracy       : 0.9842
Macro Precision: 0.9074
Macro Recall   : 0.9171
Macro F1       : 0.9087
Macro ROC-AUC  : 0.9959
```

Figure 32: Best configuration result of 1D ResNet.

So, we see from **Figure 32** that the configuration is using larger number of filter size with higher dropout rate and lower learning rate. This combination is providing better feature learning capacity while maintaining stronger regularization and more conservative optimization stage. And this tuning process highlights that the final architecture was not only selected for high test accuracy but also for its solid performance during validation.

6.3 Final Tuned 1D ResNet

With the best configuration of tuning was used to construct the final tuned 1D ResNet model shown in **Figure 33**. The parameters used here are:

a. Filters: 128
b. Kernel size: 3 and 7
c. Dropout: 0.30
d. Learning rate: 0.0005

Above configurations were used for training the final model and applied the same class-weighting method which was used during initial experiments. Because moderate class weights ensure that minor heartbeat classes can continue to receive the additional importance during training.

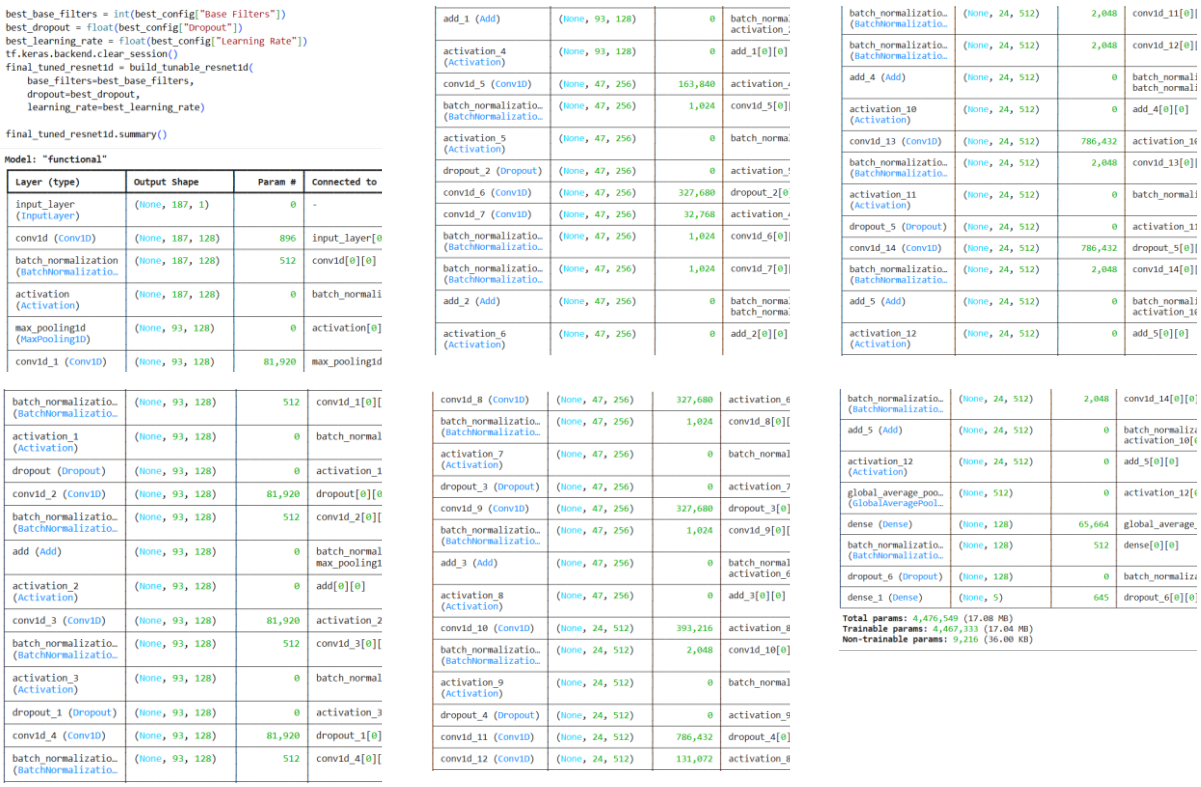


Figure 33: Building 1D ResNet with the best configuration.

The final model was not assessed with the validation set only. It was also assessed using the independent test dataset which had not been used to choose hyperparameters once the best

configuration had been found. And this way it gives more accurate assessment of the model's ability to generalize to unseen ECG heartbeat data.

6.4 Validation and Final Evaluation

In **Figure 34** the model showed rapid improvement during training. First epoch showed very poor validation, but it improved consequently during subsequent epochs. It ended with validation accuracy of 98.86% with reduced learning rate. Also, all of the evaluation metrics results are shown in **Figure 35** as proof that this model has the better performance than initial 1D ResNet architectures.

```
final_resnet_callbacks = [EarlyStopping(monitor="val_loss",patience=6,restore_best_weights=True,verbose=1),
                          ReduceLROnPlateau(monitor="val_loss",factor=0.5,patience=3,min_lr=1e-6,verbose=1)]

final_resnet_history = final_tuned_resnet1d.fit(train_ds,validation_data=val_ds,epochs=30,class_weight=class_weights,
                                                callbacks=final_resnet_callbacks,verbose=1)
```

```
Epoch 1/30
274/274 ━━━━━━━━━━━━━━━━━━━━ 59s 145ms/step - accuracy: 0.8245 - loss: 0.4976 - val_accuracy: 0.0254 - val_loss: 11.8404 - learning_rate: 5.0000e-04
Epoch 2/30
274/274 ━━━━━━━━━━━━━━━━━━━━ 27s 97ms/step - accuracy: 0.9452 - loss: 0.2319 - val_accuracy: 0.0849 - val_loss: 4.8462 - learning_rate: 5.0000e-04
Epoch 3/30
274/274 ━━━━━━━━━━━━━━━━━━━━ 27s 98ms/step - accuracy: 0.9547 - loss: 0.1892 - val_accuracy: 0.9358 - val_loss: 0.2122 - learning_rate: 5.0000e-04
.
274/274 ━━━━━━━━━━━━━━━━━━━━ 27s 99ms/step - accuracy: 0.9924 - loss: 0.0233 - val_accuracy: 0.9866 - val_loss: 0.0519 - learning_rate: 6.2500e-05
Epoch 27/30
274/274 ━━━━━━━━━━━━━━━━━━━━ 0s 93ms/step - accuracy: 0.9926 - loss: 0.0236
Epoch 27: ReduceLROnPlateau reducing learning rate to 3.1250001484295354e-05.
274/274 ━━━━━━━━━━━━━━━━━━━━ 27s 99ms/step - accuracy: 0.9926 - loss: 0.0227 - val_accuracy: 0.9872 - val_loss: 0.0480 - learning_rate: 6.2500e-05
Epoch 27: early stopping
Restoring model weights from the end of the best epoch: 21.
```

Figure 24: Code of Training Tuned 1D ResNet Model.

```
tuned_resnet_accuracy = accuracy_score(y_test,tuned_resnet_test_pred)
tuned_resnet_precision = precision_score(y_test,tuned_resnet_test_pred,average="macro",zero_division=0)
tuned_resnet_recall = recall_score(y_test,tuned_resnet_test_pred,average="macro",zero_division=0)
tuned_resnet_f1 = f1_score(y_test,tuned_resnet_test_pred,average="macro",zero_division=0)
tuned_resnet_auc = roc_auc_score(y_test,tuned_resnet_test_prob,multi_class="ovr",average="macro")

print("=" * 70)
print("FINAL TUNED RESNET1D - TEST RESULTS")
print("=" * 70)
print(f"Accuracy        : {tuned_resnet_accuracy:.4f}")
print(f"Macro Precision : {tuned_resnet_precision:.4f}")
print(f"Macro Recall    : {tuned_resnet_recall:.4f}")
print(f"Macro F1        : {tuned_resnet_f1:.4f}")
print(f"Macro ROC-AUC   : {tuned_resnet_auc:.4f}")
```

```
======================================================================
FINAL TUNED RESNET1D - TEST RESULTS
======================================================================
Accuracy        : 0.9876
Macro Precision : 0.9246
Macro Recall    : 0.9311
Macro F1        : 0.9273
Macro ROC-AUC   : 0.9940
```

Figure 34: Code and output of all evaluation metrics of Tuned 1D ResNet.

Now the final tuned 1D ResNet will be compared with the original 1D ResNet for seeing the improvement of using hyperparameter tuning. The comparison code and output shown in **Figure 35.**

```
final_accuracy = tuned_resnet_accuracy
final_precision = tuned_resnet_precision
final_recall = tuned_resnet_recall
final_f1 = tuned_resnet_f1
final_auc = tuned_resnet_auc

resnet_tuning_comparison = pd.DataFrame({
    "Model": ["Original ResNet1D","Tuned ResNet1D"],
    "Accuracy": [results.loc[results['Model'] == 'ResNet1D', 'Accuracy'].iloc[0],final_accuracy],
    "Macro Precision": [results.loc[results['Model'] == 'ResNet1D', 'Macro Precision'].iloc[0],final_precision],
    "Macro Recall": [results.loc[results['Model'] == 'ResNet1D', 'Macro Recall'].iloc[0],final_recall],
    "Macro F1": [results.loc[results['Model'] == 'ResNet1D', 'Macro F1'].iloc[0],final_f1],
    "Macro ROC-AUC": [resnet_auc,final_auc]})
resnet_tuning_comparison
```

	Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Macro ROC-AUC
0	Original ResNet1D	0.987210	0.921117	0.933186	0.926239	0.994617
1	Tuned ResNet1D	0.987621	0.924601	0.931099	0.927283	0.994048

```
improvement_df = pd.DataFrame({
    "Metric": ["Accuracy","Macro Precision","Macro Recall","Macro F1","Macro ROC-AUC"],
    "Original ResNet1D": [
        results.loc[results['Model'] == 'ResNet1D', 'Accuracy'].iloc[0],
        results.loc[results['Model'] == 'ResNet1D', 'Macro Precision'].iloc[0],
        results.loc[results['Model'] == 'ResNet1D', 'Macro Recall'].iloc[0],
        results.loc[results['Model'] == 'ResNet1D', 'Macro F1'].iloc[0],
        resnet_auc],
    "Tuned ResNet1D": [final_accuracy,final_precision,final_recall,final_f1,final_auc]})

improvement_df["Change (%)"] = (improvement_df["Tuned ResNet1D"]- improvement_df["Original ResNet1D"]) * 100

improvement_df
```

	Metric	Original ResNet1D	Tuned ResNet1D	Change (%)
0	Accuracy	0.987210	0.987621	0.041111
1	Macro Precision	0.921117	0.924601	0.348357
2	Macro Recall	0.933186	0.931099	-0.208712
3	Macro F1	0.926239	0.927283	0.104390
4	Macro ROC-AUC	0.994617	0.994048	-0.056901

Figure 35: Code and output comparison between Original and Tuned 1D ResNet.

From the output it clearly shows that the accuracy and precision and F1-score are better in tuned 1D ResNet model, which is 98.76% accuracy, 92.46% precision and 92.72% F1-score. The improvement in accuracy is slightly small it only improved from 98.72% to 98.76%. Though accuracy is not only represented by the tuning effect. The increase in precision and F1-score also supports that tuned model achieved better balance across five heartbeat categories. On the other hand, recall and ROC-AUC decreased slightly. To conclude, all of these hyperparameters tuning contains a trade-off between different prospects of classification performance rather than improving every metric simultaneously.

However, in combination with all of the metrics tuned 1D ResNet performed better than the original models that's why it will be selected as the final model for this project.

7. Visualization & Critical Analysis

7.1 Comparative Performance Analysis

The output results of four deep learning models were visualized in order to compare the tuned model with original initial models directly. In **Figure 36**, the 1D CNN, CNN-BILSTM, 1D ResNet and Tuned 1D resNet are compared, the code and outputs are given there.

```
comparison_df = pd.DataFrame({

    "Model": ["1D CNN","CNN-BiLSTM","ResNet1D","Tuned ResNet1D"],
    "Accuracy": [
        results.loc[results['Model'] == '1D CNN', 'Accuracy'].iloc[0],
        results.loc[results['Model'] == 'CNN-BiLSTM', 'Accuracy'].iloc[0],
        results.loc[results['Model'] == 'ResNet1D', 'Accuracy'].iloc[0],
        final_accuracy],
    "Macro Precision": [
        results.loc[results['Model'] == '1D CNN', 'Macro Precision'].iloc[0],
        results.loc[results['Model'] == 'CNN-BiLSTM', 'Macro Precision'].iloc[0],
        results.loc[results['Model'] == 'ResNet1D', 'Macro Precision'].iloc[0],
        final_precision],
    "Macro Recall": [
        results.loc[results['Model'] == '1D CNN', 'Macro Recall'].iloc[0],
        results.loc[results['Model'] == 'CNN-BiLSTM', 'Macro Recall'].iloc[0],
        results.loc[results['Model'] == 'ResNet1D', 'Macro Recall'].iloc[0],
        final_recall],
    "Macro F1": [
        results.loc[results['Model'] == '1D CNN', 'Macro F1'].iloc[0],
        results.loc[results['Model'] == 'CNN-BiLSTM', 'Macro F1'].iloc[0],
        results.loc[results['Model'] == 'ResNet1D', 'Macro F1'].iloc[0],
        final_f1],
    "Macro ROC-AUC": [cnn_auc,bilstm_auc,resnet_auc,final_auc]})

comparison_df
```

	Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Macro ROC-AUC
0	1D CNN	0.982962	0.883644	0.942439	0.909893	0.994978
1	CNN-BiLSTM	0.984012	0.912859	0.921870	0.917201	0.993298
2	ResNet1D	0.987210	0.921117	0.933186	0.926239	0.994617
3	Tuned ResNet1D	0.987621	0.924601	0.931099	0.927283	0.994048

Figure 35: Code and output comparison between Original Models and Tuned Model.

From the output we see that all the models have achieved great classification accuracy overall. But Tuned 1D ResNet attained slightly better test accuracy of 98.76% overall than other three original models (1D CNN: 98.29%, CNN-BiLSTM: 98.40% and 1D ResNet: 98.72%).

Though due to significant class imbalance in the dataset, others evaluation metrics like precision, recall, F1-score and ROC-AUC along with the accuracy. Considering all the metrics

then both original and tuned 1D ResNet performed better in all metrics than 1D CNN and CNN-BiLSTM, but the tuned one preformed slightly better than the original one in accuracy (0.04% better), precision (.35% better) and F1-score (0.1% better) which demonstrated that careful hyperparameter tuning can enhance model robustness and predictive quality meaningfully. Therefore, all the results support the selection of tuned 1D ResNet as the final model.

7.2 Class-Level Performance Analysis

Class level performance was investigated by using confusion matrix and classification metrics due to the highly imbalanced issue of the dataset. Confusion matrix helps to provide more informative view than accuracy only. Because it can show the categories of heartbeat which are classified correctly and which are confused with one another. It is very important for the minor class heartbeats categories as those have fewer training examples than the normal class. As the tuned 1D ResNet is selected as the final model, that's why the code and output of the confusion matrix of it is given in **Figure 36**.

```
cm_tuned_resnet = confusion_matrix(y_test,tuned_resnet_test_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_tuned_resnet,annot=True,fmt="d",cmap="Blues",xticklabels=class_names,yticklabels=class_names)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Tuned ResNet1D - Confusion Matrix")
plt.tight_layout()
plt.show()
```

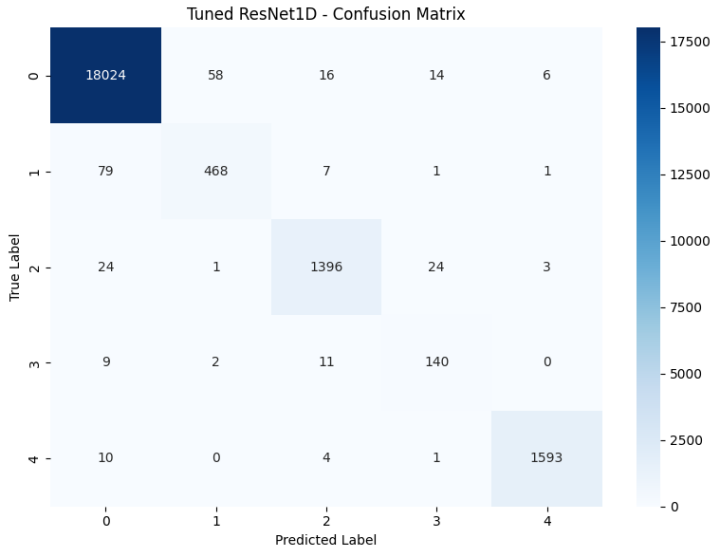


Figure 36: Code and output of Confusion Matrix of Tuned 1D ResNet.

From the matrix, it can demonstrate that the majority heartbeat data are classified correctly. Here, Class **0** contains the highest number of data and is classified with near-perfect accuracy, with few numbers of errors into other classes. The remaining classes 1 to 4 also show strong performance, with less errors which are mostly minor confusions between neighboring classes. The high diagonal values mean strong recognition of the respective heartbeat classes where off diagonal values mean the remaining are classifications error.

Also, we cross checked the classification report of the tuned model, the code and output of classification report of Tuned 1D ResNet in **Figure 37**. Class level performance is important because a model can obtain very high accuracy by correctly classifying the dominant class. The use of macro-averaged metrics reduces this effect by assigning equal importance to each class. The results demonstrate that high accuracy can prove the model performs well while precision, recall and F1-score showed that the performance is reasonably distributed across classes.

```
print("=" * 70)
print("TUNED RESNET1D CLASSIFICATION REPORT")
print("=" * 70)
print(classification_report( y_test,tuned_resnet_test_pred,target_names=list(class_names.values()),
                            digits=4,zero_division=0))
```

```
======================================================================
TUNED RESNET1D CLASSIFICATION REPORT
======================================================================
                    precision    recall  f1-score   support

         Normal (N)     0.9933    0.9948    0.9940     18118
Supraventricular (S)     0.8847    0.8417    0.8627       556
     Ventricular (V)     0.9735    0.9641    0.9688      1448
          Fusion (F)     0.7778    0.8642    0.8187       162
        Unknown (Q)     0.9938    0.9907    0.9922      1608

           accuracy                         0.9876     21892
          macro avg     0.9246    0.9311    0.9273     21892
       weighted avg     0.9877    0.9876    0.9876     21892
```

Figure 37: Code and output of Classification Report of Tuned 1D ResNet.

In short, Classification report and confusion matrix should be interpreted together for analyzing class level performance. The matrix and classification report proves that the model is highly reliable, it can maintain classified consistency and can show minimal overlap between categories which reflects the model has robust generalization and stable predictive performance.

7.3 Critical Analysis of Model Performance

7.3.1 Comparison of the Three Initial Architectures

From **Figure 29**, it can demonstrate that the performance of the three models shows that all three deep-learning models can deliver very strong and authentic results. But their strengths are different in meaningful ways. The 1D CNN is giving higher accuracy result (98.39%), but the lower precision result suggests that it struggles with minor classes slightly.

On the other hand, the CNN-BiLSTM model improves the precision (91.28%) result and maintains a balanced recall (92.18%) which means that hybrid architecture can capture temporal patterns more effectively.

Then 1D ResNet achieved the highest of all four metrics accuracy: 98.76%, precision: 92.46%, recall: 93.10% and F1-score: 92.72% which reflects that it has the ability to learn deeper and more discriminative features. This consistent results across metrics also proves that residual connections help to reduce eliminating gradients and improve generalization. Overall, the results highlight a clear progression in model capability, with ResNet1D emerging as the most robust and dependable architecture for this classification task.

7.3.2 Effects of Hyperparameters Tuning

Then the original 1D ResNet was tuned due to becoming the best performing model among three models. The tuned model achieved 98.76% from 98.72% which improved slightly. But the most important fact is that the precision improved from 92.11% to 92.46% while F1-score improved from 92.62% to 92.72% while ROC-AUC and recall decreased than the original model slightly. So, it won't be right to say tuning improved every aspect of classification. It can say like this way that tuning produced a trade-off option where it improved accuracy, precision and F1-score while sacrificing recall and ROC-AUC.

So, in analytical performance this is a solid finding than simply reporting the highest accuracy. It shows that the model selection depends on the evaluation metrices mostly. The higher precision of the tuned model is advantage for the project when the priority is decreasing the false positive predictions. Also, if identifying as many minor classes is the main priority, the higher recall of the baseline ResNet1D may be preferable.

7.4 Interpretation of ROC-AUC

From **Figure 35** we see that the ROC-AUC results were consistently very high in all four models around 99.40% to 99.49%. And **Figure 38, 39, 40 and 41** shows the ROC-AUC Curve of the four models which showing both training and validation accuracy increases quickly and stabilize near perfect performance. That demonstrates that models are capturing the inside patterns without any major issues.

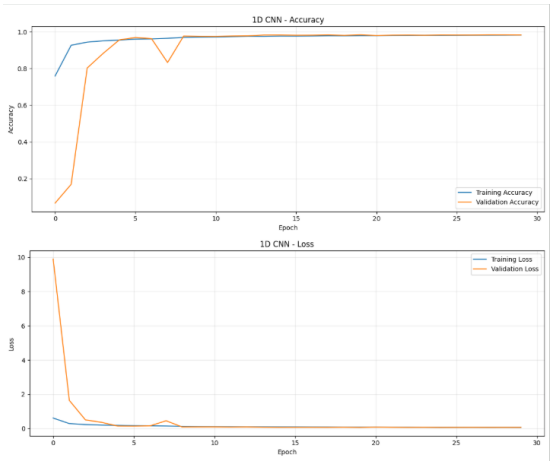


Figure 38: ROC-AUC of 1D CNN.

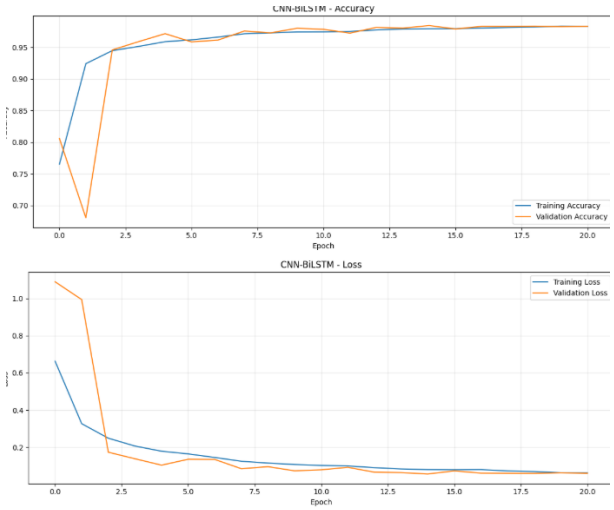


Figure 39: ROC-AUC of CNN-BiLSTM.

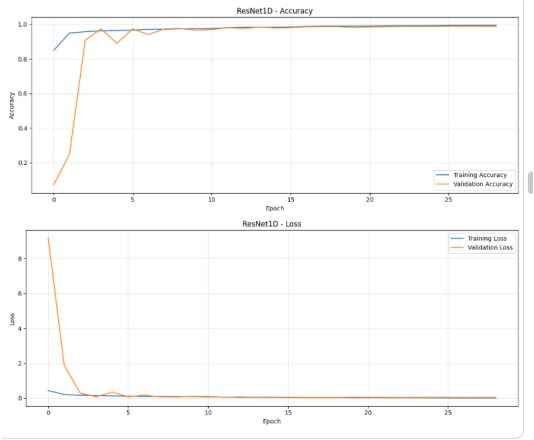


Figure 40: ROC-AUC of 1D ResNet.

The tuned model achieved the maximum score of ROC-AUC which 99.49% that indicates strong class discrimination. **Figure 35** also shows small differences between the models that suggest all structures learned with highly informative patterns of the ECH heartbeat signals.

That's why the results from tuning should not be interpreted as fundamental change in model performance but can represent as relatively small refinement of a strong model.

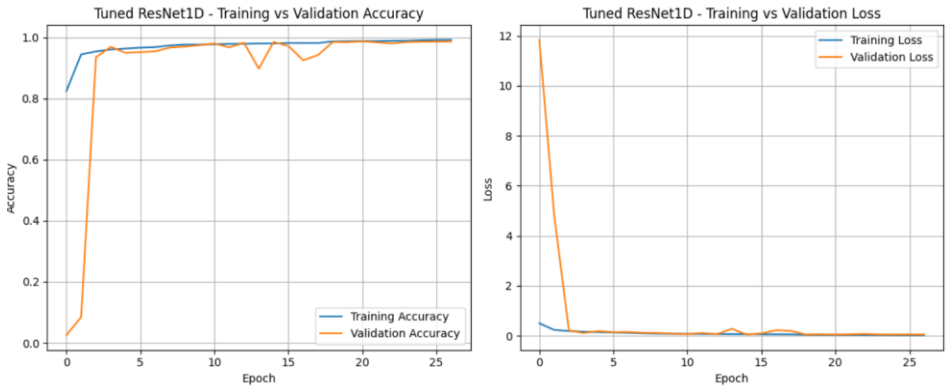


Figure 41: ROC-AUC of Tuned 1D ResNet.

It can be an important finding that the baseline models were performing their best already and there was relatively little space for improvement. So, the 0.09% improvement in ROC-AUC after tuning is valid and should not be shown as a huge performance gain.

7.5 Strengths and Limitations

7.5.1 Strengths

This study has several strengths that are listed below:

a. First strength is that all three initial models performed well and achieved high classification accuracy which supports the fact that deep learning is appropriate for the ECG heartbeat classification task.
b. It used multiple evaluation metrics that provide more balanced investigation than accuracy only. This is important as the dataset contains highly imbalanced classes.
c. Comparison between all three models: 1D CNN, CNN-BiLSTM and 1D ResNet shows that different models produce different results which provies evidence-based justification for selecting 1D ResNet for further optimization rather than choosing an architecture blindly.
d. Lastly, hyperparameter tuning experiment helped to demonstrate that the selected model can be enhanced through systematic optimization.

7.5.2 Limitations

Every project has its own strengths with limitations, and those are listed below:

a. First limitation is the dataset is highly imbalanced. Though moderate class weighting was used, minor heartbeat classes remain difficult to learn.
b. Amon all models, the performance difference is very small that's why it should not reported as evidence of 1D ResNet for being the superior to CNN and CNN-BiLSTM. So, the conclusion is limited to the experimental configuration used in this study and the dataset.
c. Final accuracy of 98.72% does not mean that the model is suitable for direct clinical use. Clinical ECG interpretation needs more factors like patient-level variation, recording conditions, noise and clinical context that are not fully presented by this experimental classification task.
d. Lastly, the framework should be considered as prototype for research of classifying automated heartbeat signals and not be used for clinical diagnostic systems.

7.6 Overall Critical Finding

All the visualization and quantitative outputs demonstrated that 1D ResNet gave the best accuracy performance overall for this study. The initial model already achieved 98.72% accuracy but after hyperparameter tuning the model achieved 98.76% accuracy which improved slightly from the initial one. The main meaningful enhancement was not in the slight increase in accuracy. It was the improvement in precision and F1-score that pointed out that the tuned model provides stronger balance among the five heartbeat categories. But here we did not isolate the fact that there was decrease in recall. The hyperparameter tuning stage involved a trade off where it improved accuracy, precision and F1-score while sacrificing recall and ROC-AUC.

Then the CNN-BiLSTM adds temporal modelling but still its results do not cross the simpler CNN that much which means bidirectional sequence learning does not necessarily translate to better results for this task. The results of CNN-BiLSTM are also informative as it explains that increasing complexity does not improve performance always.

In contrast, 1D ResNet stands out with the best initial performance at 98.72% accuracy. This model has the benefit from deeper feature extraction and residual connections. After hyperparameter tuning, 1D ResNet becomes the strongest model among other three models. It achieved 98.76% accuracy, but the most important fact is precision improved from 92.46%, F1-score 92.72% while ROC-AUC and recall decreased than the original model slightly. Considering everything still residual structure of tuned 1D ResNet appears better when the priority is decreasing the false positive predictions, higher precision of the tuned model is advantage for the project. Also, it is suited to extracting discriminative features from the one-dimensional ECG signals in this experiment.

That's why, while recognizing the limitations associated with class imbalance, dataset-specific performance and the relatively small improvement obtained through tuning. The final results support the use of tuned 1D ResNet as the final model for this study.

7. Conclusion

Here the study investigated the framework of deep learning for ECG heartbeat classification. Three different models are: 1D CNN, CNN-BiLSTM, and 1D ResNet implemented and evaluated to determine which model was the most appropriate for classifying the five heartbeat classes. The outputs explained that all three models achieved high classification accuracy that means every model has the ability of learning meaningful representations from the one-dimensional ECG signals.

1D ResNet achieved the strongest performance with 98.71% test accuracy among three initial models. As it outperformed the 1D CNN and CNN-BiLSTM. The 1D ResNet model was selected for further hyperparameter optimization. Total six configurations were checked by changing the number of filters, dropout rate, and learning rate. The best configuration consists of 128 filters, a kernel size of 3 and 7, dropout of 0.30, and a learning rate of 0.0005.

The final tuned 1D ResNet achieved 98.76% accuracy, 92.46% precision, 93.1 recall, 92.73% F1-score and 99.40% ROC-AUC. Although the improvement in accuracy over the 1D ResNet was very small but the increase in precision and F1-score indicates a better overall balance across the heartbeat classes. The results also demonstrated that a more complex CNN-BiLSTM model did not perform very well than the simpler CNN which indicates the importance of selecting a model based on the features of the data rather than complexity alone.

To conclude, the findings explain that 1D ResNet with appropriate hyperparameter optimization can be highly effective for automated ECG heartbeat classification. Though the class imbalance and dataset-specified nature of the project still are important limitations. For future work, can investigate more ECG datasets, more complex imbalance-handling methods, explainable AI methods, and patient-level validation to evaluate the robustness of the system in more realistic clinical settings.

8. References

Bai, X., Dong, X., Li, Y., Liu, R., & Zhang, H. (2024). A hybrid deep learning network for automatic diagnosis of cardiac arrhythmia based on 12-lead ECG. *Scientific Reports*, *14*(1), 24441. https://doi.org/10.1038/s41598-024-75531-w

Cacciari, I. C., & Ranfagni, A. (2026). *Hands-On Fundamentals of 1D Convolutional Neural Networks—A Tutorial for Beginner Users*. Mdpi.Com. https://www.mdpi.com/2076-3417/14/18/8500?

Hassan, S. U., Mohd Zahid, M. S., Abdullah, T. A., & Husain, K. (2022). Classification of cardiac arrhythmia using a convolutional neural network and bi-directional long short-term memory. *Digital Health*, *8*, 205520762211027. https://doi.org/10.1177/20552076221102766

Islam, M. S., Hasan, K. F., Sultana, S., Uddin, S., Lio,' P., Quinn, J. M. W., & Moni, M. A. (2023). HARDC : A novel ECG-based heartbeat classification method to detect arrhythmia using hierarchical attention based dual structured RNN with dilated CNN. *Neural Networks : The Official Journal of the International Neural Network Society*, *162*, 271–287. https://doi.org/10.1016/j.neunet.2023.03.004

Jing, E., Zhang, H., Li, Z., Liu, Y., Ji, Z., & Ganchev, I. (2021). ECG heartbeat classification based on an improved ResNet-18 model. *Computational and Mathematical Methods in Medicine*, *2021*, 1–13. https://doi.org/10.1155/2021/6649970

Khan, F., Yu, X., Yuan, Z., & Rehman, A. (2023). ECG classification using 1-D convolutional deep residual neural network. *PLOS ONE*, *18*(4), e0284791. https://doi.org/10.1371/journal.pone.0284791

Kim, N.-H., Seo, W., Kim, J., So Yoon Choi, & Sung Min Park. (2023). WavelNet: A novel convolutional neural network architecture for arrhythmia classification from electrocardiograms. *Computer Methods and Programs in Biomedicine*, *231*, 107375–107375. https://doi.org/10.1016/j.cmpb.2023.107375

Safdar, M. F., Nowak, R., & Pałka, P. (2023). Exploring artificial intelligence algorithms for electrocardiogram (ECG) signal analysis: A comprehensive review. *Computers in Biology and Medicine*, *170*, 107908–107908. https://doi.org/10.1016/j.compbiomed.2023.107908

Ullah, H., Bin Heyat, M. B., Akhtar, F., Sumbul, Muaad, A. Y., Islam, M. S., Abbas, Z., Pan, T., Gao, M., Lin, Y., & Lai, D. (2022). An end-to-end cardiac arrhythmia recognition method with an effective DenseNet model on imbalanced datasets using ECG signal. *Computational Intelligence and Neuroscience*, *2022*, 9475162. https://doi.org/10.1155/2022/9475162

Yang, X., Zhang, X., Yang, M., & Zhang, L. (2021). 12-Lead ECG arrhythmia classification using cascaded convolutional neural network and expert feature. *Journal of Electrocardiology*, *67*, 56–62. https://doi.org/10.1016/j.jelectrocard.2021.04.016

Zhou, F., & Fang, D. (2024). Multimodal ECG heartbeat classification method based on a convolutional neural network embedded with FCA. *Scientific Reports*, *14*(1), 8804. https://doi.org/10.1038/s41598-024-59311-0